

DUT-Informatique –S3

2021/2022

Introduction

Une question: Comment *gérer, mémoriser et traiter* un ensemble de données: *d'étudiants (num, nom, prénom, groupe, filière...)* ?

❖ **Première solution : utilisation des Fichiers (Excel);**

❖ Mais comment traiter les requêtes (répondre aux questions) suivantes ?

❖ **n° de Youssef Alami ?**

❖ **Les étudiants du groupe 51 ?**

❖ **La note de chaque étudiant?**

❖ • ...



Introduction

Travailler directement sur un fichier présente plusieurs **inconconvénients** :

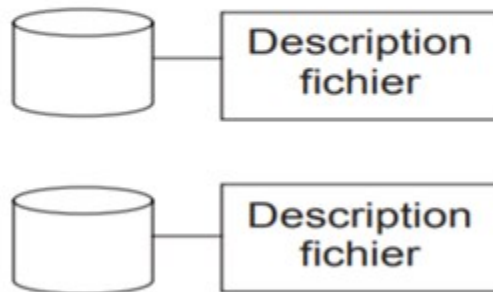
- ❖ Manipulation de données **lourde et compliquée**. Il faut être **expert** en programmation;
- ❖ Le programmeur doit connaître la **localisation physique des fichiers, la structure physique des enregistrements et le mode d'accès à ces fichiers**;
- ❖ Toute modification de la structure des enregistrements (**ajout d'un champ ou bien colonne**) entraîne la réécriture de tous les programmes qui manipulent ces fichiers.

Introduction

Séparation des données et des programmes

FICHIER

Les données des fichiers sont décrites dans les programmes



Programmes

BASE DE DONNEES

Les données de la BD sont décrites hors des programmes dans la base elle-même



Programmes

L'APPROCHE SYSTEMES DE FICHIERS

Caractéristiques

Plusieurs applications

- plusieurs formats;
- plusieurs langages;

Redondance de données

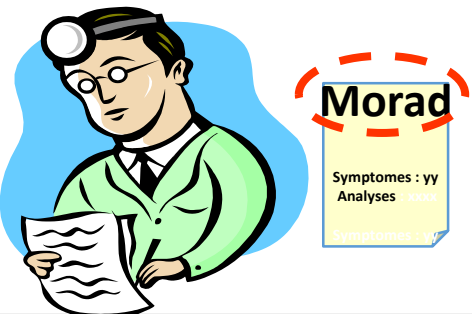
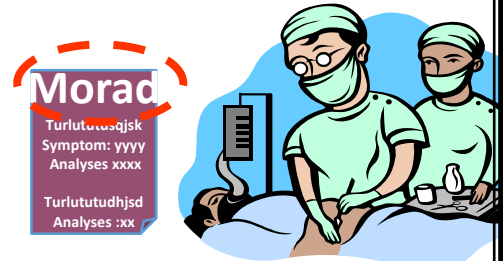
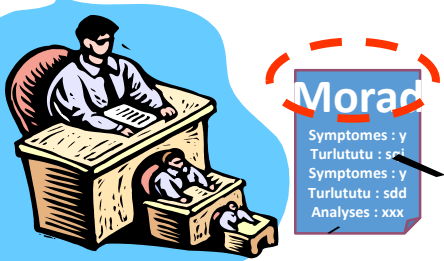
Difficulté d'interrogation

- Question $\Rightarrow$ développement;

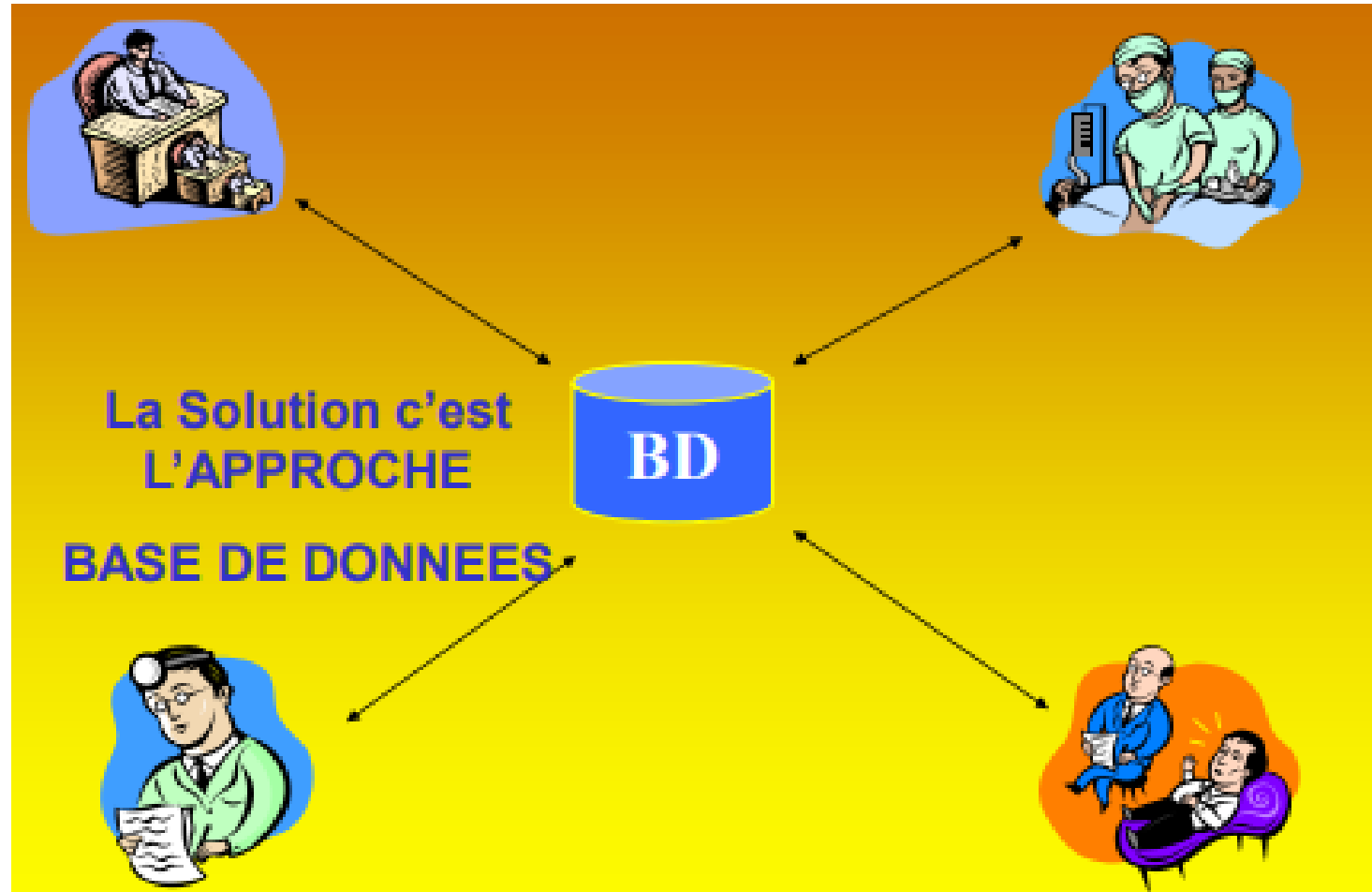
Redondance de code

Problèmes

- Difficultés de gestion;
- Incohérence des données;
- Coûts élevés;
- Maintenance difficile;
- Gestion de pannes ???
- Partage des données ???
- Confidentialité ???



Introduction





Introduction

Approche Bases de données

Une base de données est un **ensemble d'informations**, (fichiers), partagé par plusieurs **utilisateurs (ou bien application)**. Ces informations **sont interrogées et mises à jour par l'intermédiaire d'un logiciel (SGBD)**.

- ❖ Modélisation des données (physique) ;
- ❖ **Eliminer les redondances** de données ;
- ❖ Centraliser, répartir et organiser correctement les données ;
- ❖ Sécurité, contrôle et gestion de la base de données;
- ❖

Introduction

Objectifs d'une base de données

Une base de données doit assurer les fonctionnalités suivantes:

- ❖ Bonne représentation du monde réel;
- ❖ Simplicité de contrôle et de gestion de données;
- ❖ **Non redondance de l'information;**
- ❖ Assure l'Indépendance entre **les programmes et les données;**
- ❖ Sécurité et la gestion de la base de données;
- ❖ Partage des données;
- ❖ Cache toute complexité **logique** ou bien **physique**;
- ❖

Introduction

Avantages des bases de données

- ❖ **Intégration des données** : La centralisation des données dans une base de données facilite bien leur gestion et leur contrôle. Plus l'information est dispersée, plus **elle est redondante et moins elle est contrôlée**.
- ❖ **Contrôle de redondance** : La redondance est la présence de la **même donnée** dans des endroits différents dans un **système d'information**.
- ❖ **La cohérence**: Une base de données est cohérente si elle ne contient pas des données **contradictaires** fournit la même réponse à une requête formulée de différentes façons ou exécutées plusieurs fois. **L'incohérence peut être due à la présence des redondances**.

Introduction

Avantages des bases de données

- ❖ **L'intégrité**: L'intégrité signifie que les données stockées dans la base sont **exactes** ou **correctes**. L'incohérence entre deux valeurs de la même donnée est une perte d'intégrité
- ❖ **Partage de la base entre plusieurs utilisateurs** : le SGBD permet aux utilisateurs d'accéder simultanément à la même base de données (**accès concurrent**),
- ❖ **Sécurité** : les accès à la base (lecture/écriture) sont protégés par le SGBD. Un accès non autorisé ne peut être effectué. En plus, les données centralisées sont faciles à protéger que les données dispersées.

Introduction

Avantages des bases de données

- ❖ **La disponibilité:** La reprise après panne permise par les SGBD assure la disponibilité des données qui est un aspect de sécurité.
- ❖ **Distribution des données :** la base de données peut être partitionnée sur plusieurs sites, sur diverses plateformes. Ce qui peut **améliorer** les **performances** du système.
- ❖

Généralités et Concepts de Base

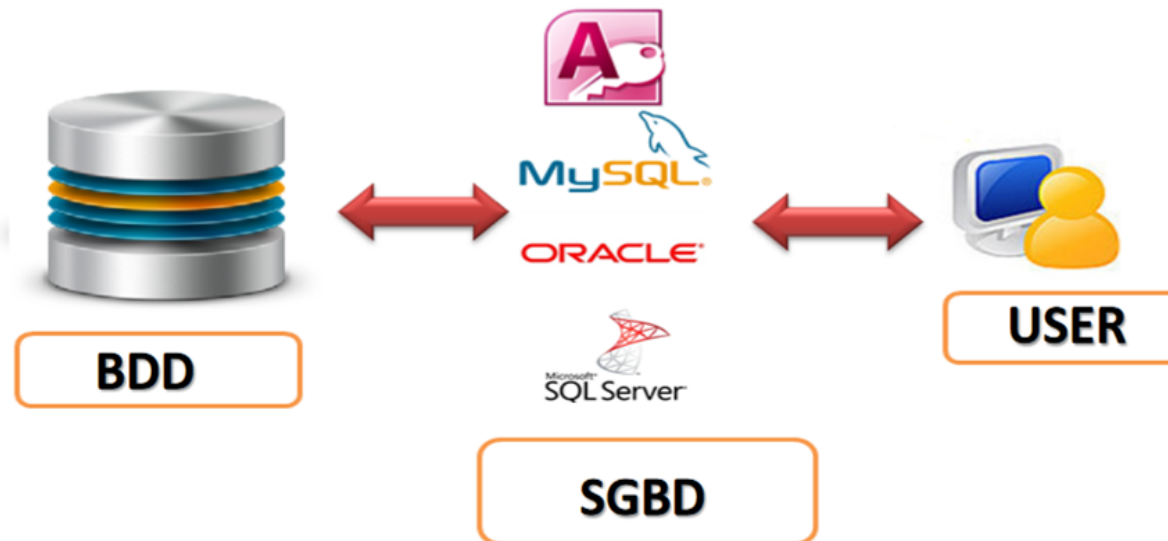
- ❖ **Une requête est une expression d'opérations de l'algèbre** relationnelle. Elle permet d'élaborer les réponses à la plupart des questions que l'on peut poser à une base de données relationnelle.
- ❖ Autrement dit : **Un ensemble d'instructions SQL se nomme une requête.**
- ❖ Une donnée est statique. Elle peut être **temporaire** ou **permanente**. Elle rentre dans un programme, subit des traitements, participe à des traitements et peut être conservée ou détruite après traitement.

Généralités et Concepts de Base

- ❖ Une base de données relationnelle est un ensemble exhaustif de données structurées qui sont **fiables, cohérentes et partagées** par des utilisateurs ayant des besoins d'information différentes.
- ❖ Un modèle est un outil qui permet de dériver l'abstraction d'une entreprise.

Généralités et Concepts de Base

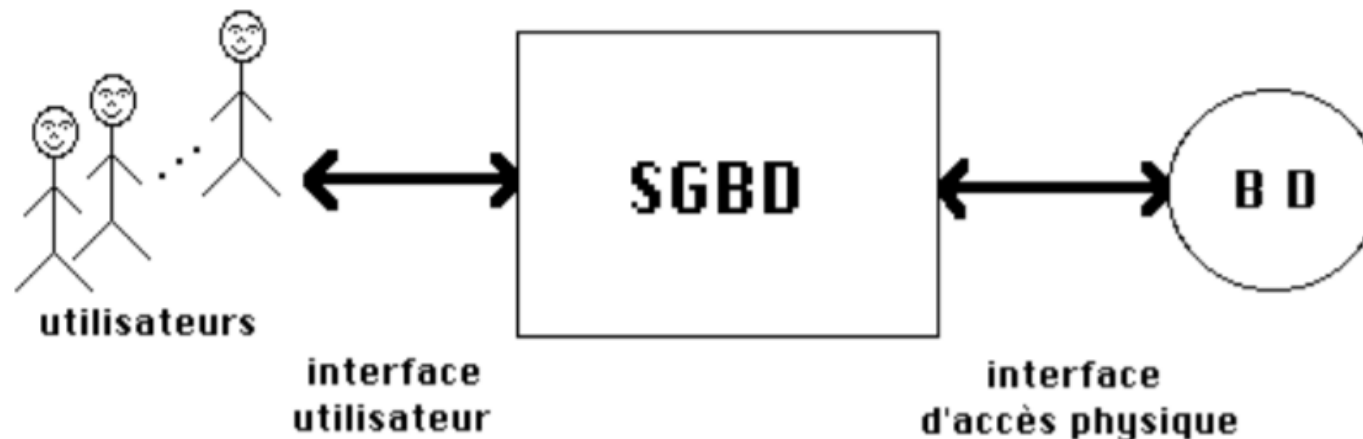
Le SGBD(logiciel) joue le rôle **d'intermédiaire entre l'utilisateur et les données dans la base de données**. En effet l'utilisateur exprime sa requête (demande) en utilisant un langage simple SQL et la présente au SGBD. Ce dernier interprète et exécute les instructions qu'il contient ou envoie un message d'erreurs.



Généralités et Concepts de Base

Pour assurer la gestion d'une base de données, Le SGBD(logiciel) définit deux interfaces:

- ❖ **L'interface utilisateur** permet aux utilisateurs d'exprimer des requêtes;
- ❖ **L'interface d'accès physique** permet au SGBD d'accéder aux données sur les supports (disques, ...).



Généralités et Concepts de Base

Un SGBD **permet de :**

- ❖ Définir une base de données (Create, Add, Alter, Drop...).
- ❖ Manipuler une base de données:
 - ❖ Accès en lecture (Select);
 - ❖ Accès en mise à jour (Update) ;
 - ❖ Insertion (Insert);
 - ❖ Modification (Modify);
 - ❖ Suppression (Delete).
- ❖ Contrôler une base de données (Grant et Revoke).



Généralités et Concepts de Base

Autrement; il représente un ensemble coordonné de logiciels qui permet **de décrire, mémoriser, manipuler, traiter, interroger** les ensembles de données constituant la base. Il assure aussi les fonctionnalités suivantes:

- ❖ Manipulation des données;
- ❖ Contrôler la redondance d'informations;
- ❖ Intégrité des données;
- ❖ Partage des données;
- ❖ Confidentialité des données;
- ❖ Sécurité de fonctionnement;
- ❖ Support de persistance de données.

Généralités et Concepts de Base

Dictionnaire de base de données

Chaque base de données possède un dictionnaire qui contient les **définitions** et **des informations sur les objets de cette base de données (entités, utilisateurs, contraintes, privilèges, etc.)**.

- ❖ Il est en soi une base de données **appelée Meta-base**.
- ❖ Il doit respecter les contraintes suivantes :
 - ❖ Tous les **noms** doivent être **mono-valués** et **non décomposables**.
 - ❖ Les données sont **regroupées** par des **entités**.
 - ❖ Les **identifiants** sont **complètement précisés**.
 - ❖ Les **commentaires** doivent être **pertinents**.

Généralités et Concepts de Base

Schéma d'une base données

D'une base de données, un **schéma** définit sa structure en termes:

- ❖ de tables,
- ❖ de colonnes
- ❖ de clés primaires et de clés étrangères.

FOURNISSEUR		
NUMF	NOMF	VILLEF

PIECE	
NUMP	TYPE

OFFRE		
NUMFL	NUMPL	PRIX



Modélisation et Normalisation

La **modélisation** consiste à **créer une représentation** virtuelle d'une réalité de telle façon à faire ressortir les points auxquels on s'intéresse.

- ❖ Eliminer la **redondance** de données;
- ❖ **Centraliser** et **organiser** correctement les données;
- ❖ Niveaux de modélisation;
- ❖ Outils de conception.

La **normalisation** élimination des situations qui induisent des **Redondances et des Valeurs NULL** :

- ❖ Elle décompose une relation non normalisée en un ensemble équivalent de relations normalisées (1FN, 2FN...).

Modélisation du réel

Relation ou table

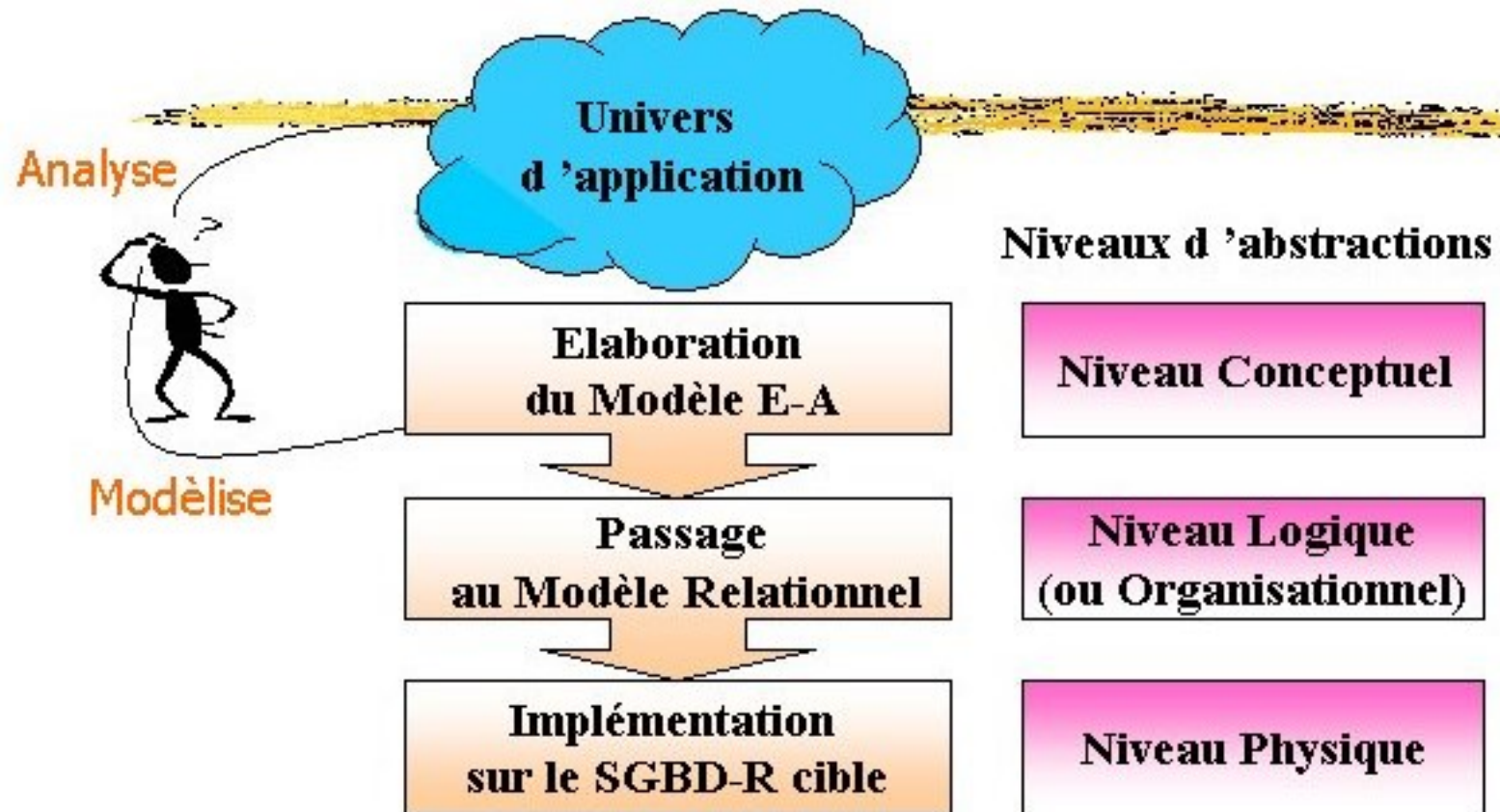
Champs, attributs, colonnes

Etudiant

Id-D	Nom	Prénom
1	Dupont	Pierre
2	Durand	Paul
3	Masse	Jean
....		

Tuples, lignes ou n-uplets

Conception d'une base de données relationnelle



Le Modèle Conceptuel

Il permet la description conceptuelle des structures de données d'une application. Il consiste à concevoir le SI sans aucun lien avec l'organisation du point de vue données ou traitements. Il traduit les deux aspects **–statique et dynamique– de la base de données**. Il se fonde sur plusieurs méthodes et outils d'aide à la conception des bases de données (MERISE).

- ❖ Entités (objets);
- ❖ Associations (liens);
- ❖ Attributs (propriétés);
- ❖ Remarques:
 - ❖ Ce Modèle donne une description de la base de données indépendamment des **choix techniques** d'implantation.
 - ❖ Il correspond à la **phase d'analyse** qui aboutit à la détermination d'une base de données.



Le Modèle LOGIQUE

Il permet l'organisation des données dans les bases de données relationnelles. Il représente un choix logiciel pour le système d'information. Il traduit le modèle conceptuel de données (MCD) vers le modèle logique de données (MLD). Il se fonde sur ces principaux concepts:

- ❖ Relation/Table;
- ❖ Identifiant/Clé primaire;
- ❖ Identifiant externe/Clé étrangère;
- ❖ Domaine.

Remarques:

- ❖ Pour la construction du Modèle LOGIQUE, on suit les règles de passage du MCD vers le MLD.
- ❖ Une table est un tableau à deux dimensions qui contient un ensemble des lignes.

Le Modèle Physique

Il permet l'implantation d'une base de données relationnelle dans un langage du SGBD. Effectivement, il modélise **la structure physique générale** d'une base de données, en tenant compte des considérations logicielles ou des contraintes relatives au stockage des données. Ces principaux concepts sont:

- ❖ Table;
- ❖ Colonnes et lignes;
- ❖ Contraintes d'intégrité;
- ❖ Clé primaire et Clé étrangère;
- ❖ Index, Vues, rôles....
- ❖

Généralités et Concepts de Base

Cycle de vie d'une base de données:

Un ouvrage est une oeuvre littéraire publiée. Il est caractérisé par son numéro identifiant, son titre, son éditeur, sa date de première parution, ses mots-clés (10 au maximum), une brève note de présentation (ces notes sont en cours de constitution), le nom et le prénom de ses auteurs. A un ouvrage correspondent un certain nombre d'exemplaires, qui en sont la matérialisation physique. ...

Conception de
base de données

```
create database BIB
create dbspace BIB_DATA;
create table OUVRAGE (
    NUMERO char(18) not null,
    TITRE varchar(60) not null,
    EDITEUR char(32) not null,
    DATE_1RE_PARUTION date not null,
    PRESENTATION varchar(255),
    primary key (NUMERO)) in BIB_DATA;
...
alter table EXEMPLAIRE add constraint FKDE
foreign key (NUMERO) references OUVRAGE;
...
create unique index IDOUVRAGE
on OUVRAGE (NUMERO);
...
```

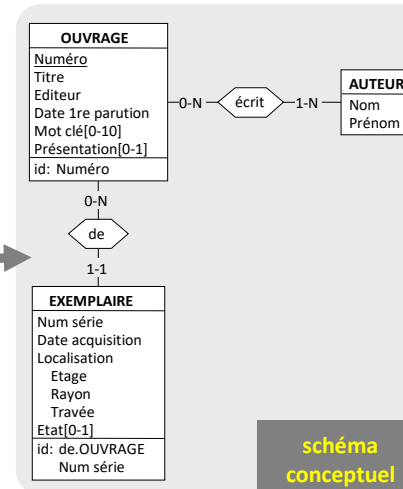


schéma
conceptuel

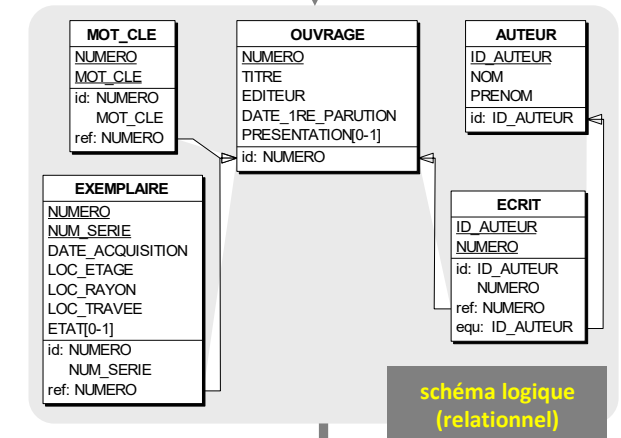


schéma logique
(relationnel)

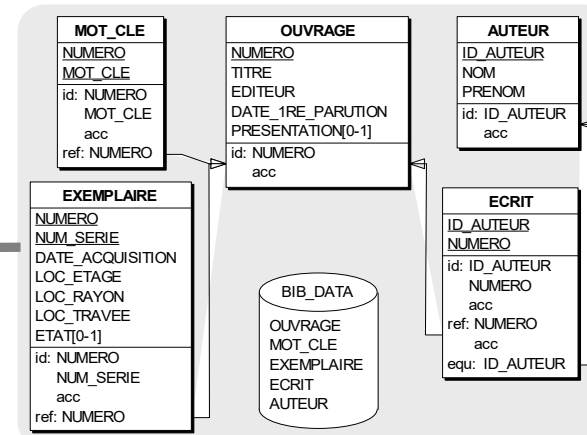
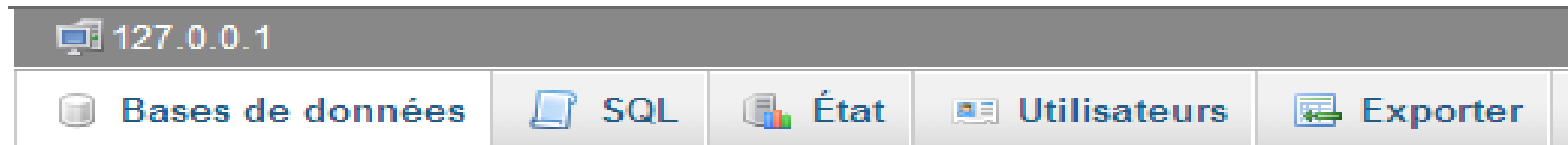


schéma physique
(Oracle 11)

Généralités et Concepts de Base

Exemple d'une base de données sous MySQL



Bases de données

 Créer une base de données 

Interclassement



Créer

Base de données ▲ Réplication maître



asimi



Répliqué



Vérifier les privilèges

Total: 1

Généralités et Concepts de Base

127.0.0.1 » asimi

Structure SQL Rechercher Requête Exporter Importer Opérations plus

Table	Action	Lignes	Type	Interclassement	Taille	Perte
0 table(s)	Somme	0	InnoDB	latin1_swedish_ci	0 o	0 o

↑ Tout cocher / Tout décocher Pour la sélection :

Version imprimable Dictionnaire de données

 Nouvelle table

Nom: Nombre de colonnes:

Exécuter

Généralités et Concepts de Base

Nom de la table: Ajouter colonne(s)

Structure									
Nom	Type	Taille/Valeurs*	Défaut	Interclassement	Attributs	Null	Index	A_I	Commentaire
ID	INT		Aucune	armSCII8_bin		☐	PRIMARY	☒	
NOM	VARCHAR	40	Aucune	armSCII8_bin		☐	---	☐	
PRENOM	VARCHAR	40	Aucune	armSCII8_bin		☐	---	☐	

Commentaires sur la table:

Moteur de stockage:

Interclassement:

Définition de PARTITION:

Généralités et Concepts de Base

127.0.0.1 » asimi » client

Afficher Structure SQL Rechercher Insérer Exporter Importer plus

✓ MySQL a retourné un résultat vide (aucune ligne). (Traitement en 0.0004 sec)

```
SELECT *
FROM `client`
LIMIT 0 , 30
```

☐ Profilage [En ligne] [Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]

#	Nom	Type	Interclassement	Attributs	Null	Défaut	Extra	Action
☐ 1	ID	int(11)			Non	Aucune	AUTO_INCREMENT	Modifier Supprimer ▼ plus
☐ 2	NOM	varchar(40)	armscii8_bin		Non	Aucune		Modifier Supprimer ▼ plus
☐ 3	PRENOM	varchar(40)	armscii8_bin		Non	Aucune		Modifier Supprimer ▼ plus

↑ Tout cocher / Tout décocher Pour la sélection : Afficher Modifier Supprimer Primaire Unique Index

Version imprimable vue relationnelle Suggérer des optimisations quant à la structure de la table Suivre la table

Ajouter colonne(s) ☒ En fin de table ☐ En début de table ☐ Après

+ Index

Généralités et Concepts de Base

The screenshot displays a web-based database management interface. At the top, the address bar shows '127.0.0.1 » asimi » client'. Below the address bar is a navigation menu with buttons: 'Afficher', 'Structure', 'SQL', 'Rechercher', 'Insérer', 'Exporter', 'Importer', and a 'plus' dropdown. A green success message banner states: '2 lignes insérées. Identifiant de la ligne insérée : 2'. Below this, the SQL query is shown in a text area:

```
INSERT INTO `asimi`.`client` (  
  `ID`,  
  `NOM`,  
  `PRENOM`  
)  
VALUES (  
  NULL, 'ASIMI', 'YOUNES'  
) , (  
  NULL, 'TBATOU', 'ZAKARIAE'
```

 At the bottom right of the query area are links: '[En ligne] [Modifier] [Créer source PHP]'. Below the query area is a section titled 'Exécuter une ou des requêtes SQL sur la base asimi:'. It contains a text input field with the full SQL statement:

```
INSERT INTO `asimi`.`client` (`ID`, `NOM`, `PRENOM`) VALUES  
(NULL, 'ASIMI', 'YOUNES'), (NULL, 'TBATOU', 'ZAKARIAE');
```

 To the right of the input field is a 'Colonnes' (Columns) list containing: 'ID', 'NOM', and 'PRENOM'.

Généralités et Concepts de Base

The screenshot shows a web-based database management tool interface. At the top, there's a navigation bar with icons and labels for 'Afficher', 'Structure', 'SQL', 'Rechercher', 'Insérer', 'Exporter', 'Importer', and a 'plus' dropdown. Below this, a green status bar indicates 'Affichage des lignes 0 - 1 (~2 total) ? , Traitement en 0.0005 sec)'. The main area displays a SQL query: `SELECT * FROM 'cleint' LIMIT 0 , 30`. Below the query, there are links: 'Profilage [En ligne] [Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]'. A control bar shows 'Afficher : Ligne de départ: 0 Nombre de lignes: 30 En-têtes à chaque 100 ligne'. Below this, a dropdown menu for 'Trier sur l'index:' is set to 'Aucune'. A section titled '+ Options' contains a table with two rows of data. The first row has columns for checkboxes, actions (Modifier, Copier, Effacer), and values (1, asimi, younes). The second row has similar data (2, Tbatou, Zakariae). At the bottom, there's a summary bar with 'Afficher : Ligne de départ: 0 Nombre de lignes: 30 En-têtes à chaque 100 ligne' and a selection bar with 'Tout cocher / Tout décocher' and actions for 'Modifier', 'Effacer', and 'Exporter'.

127.0.0.1 » asimi » cleint

Afficher Structure SQL Rechercher Insérer Exporter Importer plus

✓ Affichage des lignes 0 - 1 (~2 total) ? , Traitement en 0.0005 sec)

```
SELECT *  
FROM 'cleint'  
LIMIT 0 , 30
```

☐ Profilage [En ligne] [Modifier] [Expliquer SQL] [Créer source PHP] [Actualiser]

Afficher : Ligne de départ: 0 Nombre de lignes: 30 En-têtes à chaque 100 ligne

Trier sur l'index: Aucune

+ Options

			ID	NOM	PRENOM
☐	Modifier	Copier	Effacer	1	asimi younes
☐	Modifier	Copier	Effacer	2	Tbatou Zakariae

↑ Tout cocher / Tout décocher Pour la sélection : Modifier Effacer Exporter

Afficher : Ligne de départ: 0 Nombre de lignes: 30 En-têtes à chaque 100 ligne

Généralités et Concepts de Base

L'algèbre relationnelle est à l'origine du langage **SQL, langage d'interrogation, de contrôle et de manipulation** de tous les SGBDR actuels (**Oracle**, PostgreSQL, MySQL...). Il assure les fonctionnalités suivantes :

- ❖ la création des objets de bases de données et les contraintes associées ;
- ❖ l'interrogation et la maintenance des bases de données ;
- ❖ la sécurité des bases de données.
- ❖ Il a trois **dimension (LDD, LMD et LCD)**.

1) Langage de définition des données (LDD):

- ❖ Il décrit le schéma conceptuel (structure) de la base de donnée.
- ❖ Il permet aussi de créer ou supprimer des relations et des attributs.
- ❖ Il contient des commandes pour **créer, modifier ou effacer** la définition (schéma) d'une BD ou d'une table.

Généralités et Concepts de Base

2) *Langage de manipulation des données (LMD):*

- ❖ *Il donne accès aux données de la BD pour la **consulter ou la modifier** à l'aide d'opérations.*
- ❖ *Il permet de lire, créer, supprimer ou modifier les enregistrements dans les relations (**select, insert, update....**).*

3) *Langage de contrôle des données (LCD):*

- ❖ *Il assure le **contrôle d'accès aux données et la gestion des privilèges**;*

Généralités et Concepts de Base

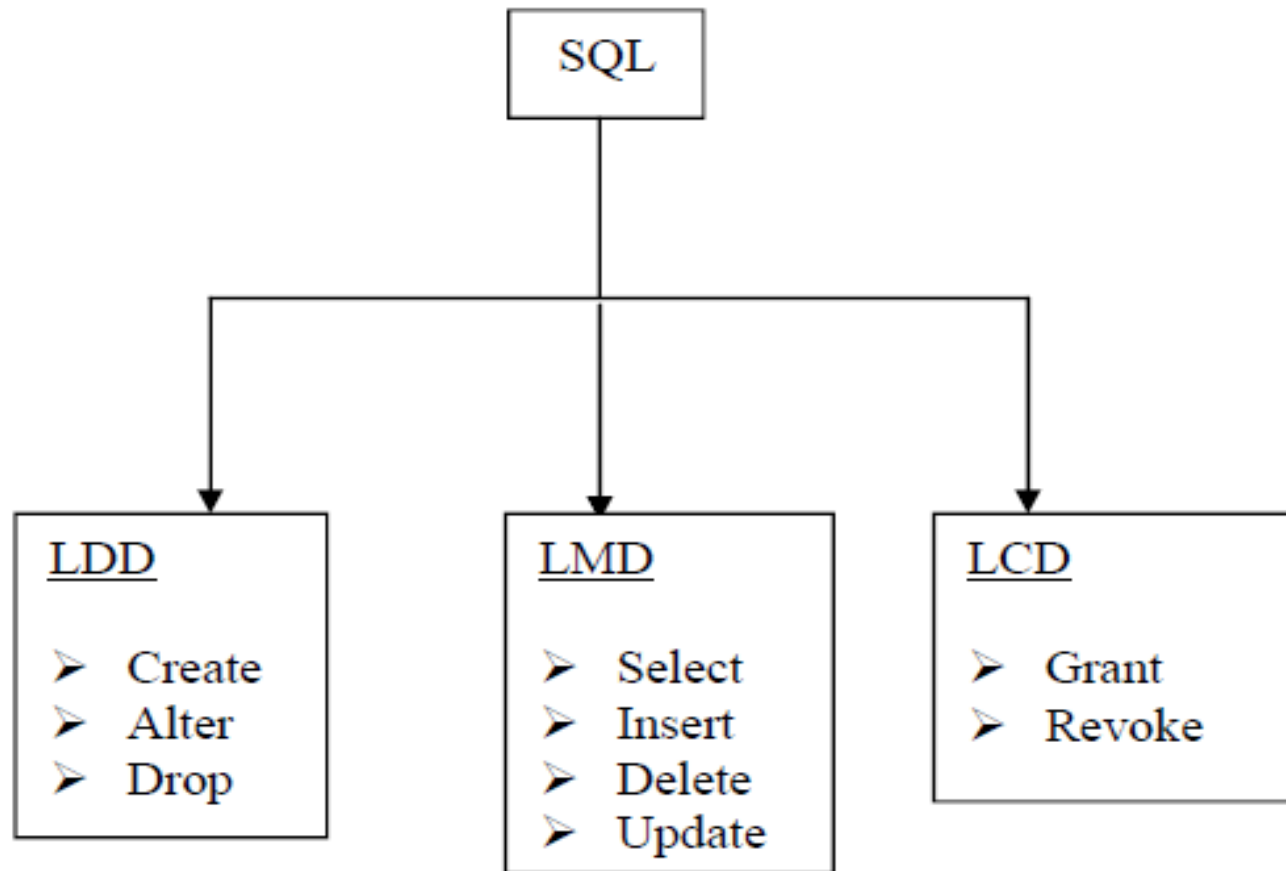
Privilèges globaux (Tout cocher / Tout décocher)

Veuillez noter que les noms de privilèges sont exprimés en anglais

Données	Structure	Administration	Limites de ressources
☒ SELECT	☒ CREATE	☒ GRANT	<i>Note: Une valeur de 0 (zero) enlève la limite.</i>
☒ INSERT	☒ ALTER	☒ SUPER	MAX QUERIES PER HOUR 0
☒ UPDATE	☒ INDEX	☒ PROCESS	MAX UPDATES PER HOUR 0
☒ DELETE	☒ DROP	☒ RELOAD	MAX CONNECTIONS PER HOUR 0
☒ FILE	☒ CREATE TEMPORARY TABLES	☒ SHUTDOWN	MAX USER_CONNECTIONS 0
	☒ SHOW VIEW	☒ SHOW DATABASES	
	☒ CREATE ROUTINE	☒ LOCK TABLES	
	☒ ALTER ROUTINE	☒ REFERENCES	
	☒ EXECUTE	☒ REPLICATION CLIENT	
	☒ CREATE VIEW	☒ REPLICATION SLAVE	
	☒ EVENT	☒ CREATE USER	
	☒ TRIGGER		

Exécuter

Généralités et Concepts de Base



Généralités et Concepts de Base

Contraintes d'intégrité

Elles permettent la **simplification** du code des applications et **renforcement de la sécurité** par la **vérification automatique de ces contraintes**, plutôt, il exigent un langage de **définition des contraintes d'intégrité**:

- ❖ Définir un état **cohérent** de la base de données;
- ❖ Vérifier **automatiquement** à chaque **insertion**, **modification** ou **suppression** les données.
- ❖ Il existe **trois contraintes d'intégrité** :
 - ❖ **Contrainte de Domaine**: Les valeurs d'un attribut doivent appartenir à son domaine;
 - ❖ **Contrainte de Relation**: Chaque relation possède **une clé primaire unique et non nulle**. La clé primaire est soulignée dans un schéma de relation.
 - ❖ **Contrainte de Référence**: La valeur d'une **clé étrangère dans la relation référençant** doit figurer comme valeur de la **clé primaire associée dans la relation référencée**.



Cours
Introduction aux Bases de Données
Ecole Supérieure de Technologie – Guelmim
Université Ibn Zohr – Agadir-
DUT-Informatique – S3

Prof. ASIMI Younes
asimi.younes@gmail.com

2021/2022

Généralités et Concepts de Base

Contraintes d'intégrité

Elles permettent la **simplification** du code des applications et **renforcement de la sécurité** par la **vérification automatique de ces contraintes**, plutôt, il exigent un langage de **définition des contraintes d'intégrité**:

- ❖ Définir un état **cohérent** de la base de données;
- ❖ Vérifier **automatiquement** à chaque **insertion**, **modification** ou **suppression** les données.

Il existe **trois contraintes d'intégrité** :

- ❖ **Contrainte de Domaine**: Les valeurs d'un attribut doivent appartenir à son domaine;
- ❖ **Contrainte de Relation**: Chaque relation possède **une clé primaire unique et non nulle**. La clé primaire est soulignée dans un schéma de relation.
- ❖ **Contrainte de Référence**: La valeur d'une **clé étrangère dans la relation référençant** doit figurer comme valeur de la **clé primaire associée dans la relation référencée**.

Généralités et Concepts de Base

Contraintes d'intégrité

- ❖ Sur les **colonnes** :
 - ❖ **NOT NULL** : Déclarer une colonne obligatoire;
 - ❖ **DEFAULT** : Préciser une valeur par défaut;
 - ❖ **UNIQUE** : Vérifier que toutes les valeurs sont différentes;
 - ❖ **CHECK** : **Contrôle la validité** de la valeur des colonnes (**Intégrité de domaine**);
 - ❖ **CONSTRAINT** : permet d'ajouter et de renommer une contrainte sur une colonne donnée d'une table;
- ❖ Sur la **table** globalement :
 - ❖ **CONSTRAINT.....**
 - ❖ **PRIMARY KEY** : Préciser la clé primaire d'une table donnée;
 - ❖ **FOREIGN KEY** (col1) **REFERENCES** NamTable([col2]),: Préciser la/les clés étrangères => **intégrité référentielle**.



Généralités et Concepts de Base

Contraintes d'intégrité

Afin d'éviter le problème d'intégrité de données, on doit suivre les étapes suivantes:

- ❖ Bien définir les **domaines de données** lors de la création de la structure d'une table;
- ❖ Lors de la création, il faut commencer par les tables qui ne contiennent pas des **contraintes de référence**;
- ❖ Puis, nous pouvons construire les **tables fils (FK)** à condition de vérifier l'existence de **la table père (PK)**;
- ❖ Cocher l'application de **l'intégrité référentielle** lors de la mise en place des liaisons.
- ❖ Saisir en premier les données des tables qui ne contiennent pas des **clés étrangères**;

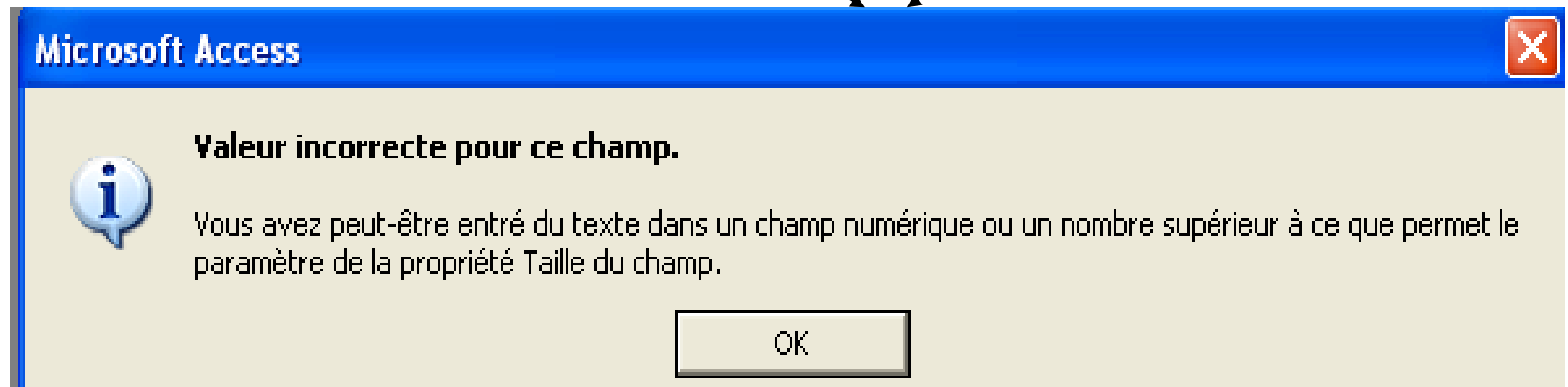
Généralités et Concepts de Base

Contraintes d'intégrité

❖ Contrainte de **domaine**:

		refprod	desiprod	marqueprod	prixprod
/	+	RF245	REFRIGERATE	MIELE	non connu €

Nom du champ	Type de données
prixprod	Monétaire



Généralités et Concepts de Base

Contraintes d'intégrité

❖ Contrainte de **relation**:

	refprod	desiprod	marqueprod	prixprod
	+ CG4921	CONGELATEUR	MOULINEX	200,00 €
	+ CG5327	CONGELATEUR	MIELE	215,00 €
	+ CS1572	CUISINIERE	MOULINEX	790,00 €
	+ CS547	CUISINIERE	ARTHUR MARTIN	950,00 €
✓	+ CG5327	CONGELATEUR	MIELE	215,00 €

Microsoft Access



Modifications non effectuées: risque de doublons dans champs index, clé principale ou relation interdisant les doublons. Modifiez les données des champs contenant les doublons, enlevez ou redéfinissez l'index pour permettre les doublons et recommencez.

OK

Aide

Généralités et Concepts de Base

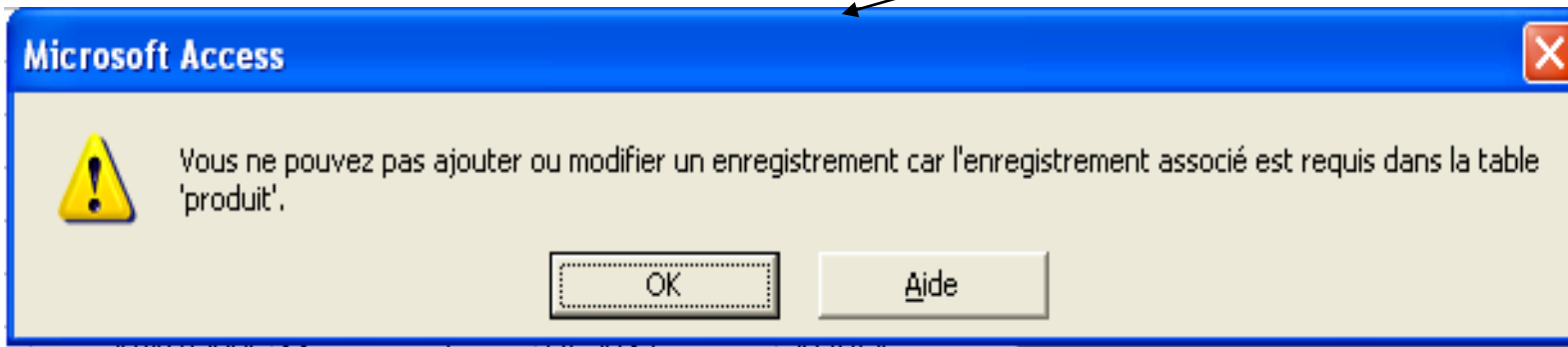
Contraintes d'intégrité

❖ Contrainte de **référence**:

	coderecrec	daterec	raisonrec	refprod	numcli
	27	17/12/2006	Notice	CS1547	421072

Modification du champ
refprod est interdite

???



refprod	desiprod	marqueprod	prixprod
CG2054	CONGELATEUR	MOULINEX	245,00 €
CG4210	CONGELATEUR	BRANDT	229,00 €
CG4921	CONGELATEUR	MOULINEX	200,00 €
CG5327	CONGELATEUR	MIELE	215,00 €
CS1572	CUISINIERE	MOULINEX	790,00 €
CS547	CUISINIERE	ARTHUR MARTIN	950,00 €
CS7814	CUISINIERE	ROSIERES	900,00 €
LL1043	LAVE LINGE	BRANDT	610,00 €
LL1050	LAVE LINGE	MOULINEX	520,00 €
LL450	LAVE LINGE	ARTHUR MARTIN	250,00 €
LV2050	LAVE VAISSELLE	BRANDT	1 035,00 €
LV5790	LAVE VAISSELLE	MIELE	1 050,00 €
RF147	REFRIGERATEUR	MIELE	750,00 €
RF2047	REFRIGERATEUR	BRANDT	1 020,00 €
RF243	REFRIGERATEUR	WHIRLPOOL	990,00 €

Généralités et Concepts de Base

Confidentialité

- ❖ Le but est de **Protéger les données de la BD** contre des accès non autorisés.
- ❖ Deux niveaux :
 - ❖ Connexion restreinte aux **usagers répertoriés** (**mot de passe**);
 - ❖ **Privilèges** d'accès aux objets de la base de données.

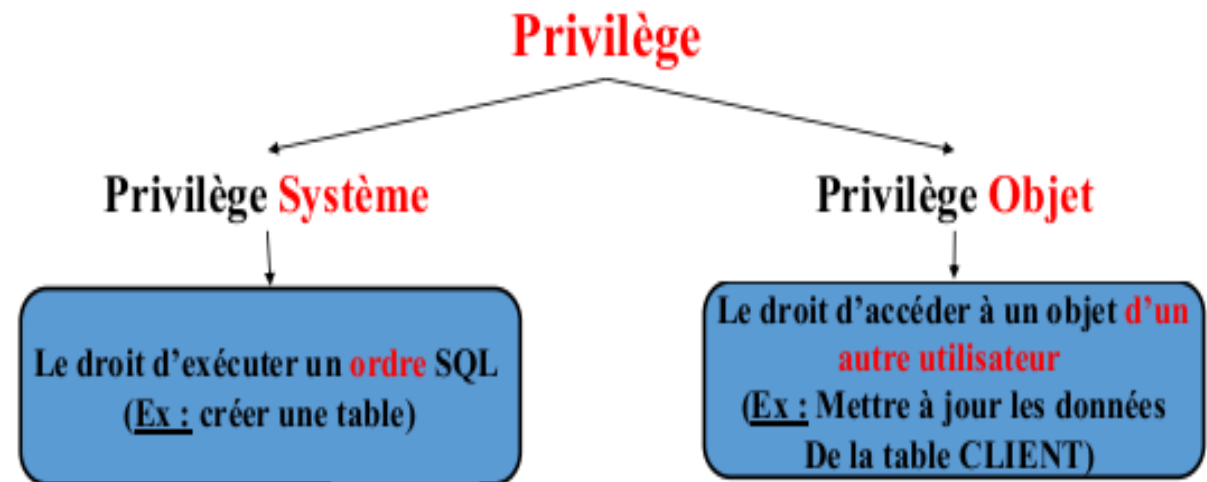
Généralités et Concepts de Base

Privilèges

Le **privilège** est défini comme la délégation d'autorité sur un système informatique. Un privilège permet à un utilisateur d'effectuer une action. Parmi les différents privilèges, citons la possibilité de :

- ❖ Créer un fichier dans un dossier;
- ❖ Lire ou de supprimer un fichier;
- ❖ Accéder à un périphérique;
- ❖ Installer un logiciel;
- ❖ Ouvrir une session par un utilisateur;
- ❖ Exécuter une requête SQL ;
- ❖ Créer une base de données;
- ❖ Gérer les tables dans une base de données;

Nous distinguons deux types de privilèges:



Généralités et Concepts de Base

Privilèges Système – Niveau Administrateur

L'administrateur de la base de données a la plupart des privilèges permettant, entre autre, de :

- ❖ Créer de nouveaux utilisateurs ;
- ❖ Supprimer des utilisateurs ;
- ❖ Supprimer des tables ;
- ❖ Récupérer les tables.....

Privilège système	Operations autorisées
CREATE USER	Permet de créer d'autres utilisateurs Oracle
DROP USER	Permet de supprimer d'autres utilisateurs
DROP ANY TABLE	Permet de supprimer une table dans n'importe quel schéma
BACKUP ANY TABLE	Permet de sauvegarder et de restaurer n'importe quelle table dans n'importe quel schéma avec l'utilitaire d'export
SELECT ANY TABLE	Permet d'interroger les tables, les vues ou les vues matérialisées dans n'importe quel schéma
CREATE ANY TABLE	Permet de créer des tables dans n'importe quel schéma

Généralités et Concepts de Base

Privilèges Système – Niveau utilisateur

Une fois que le DBA ait créé un utilisateur, il peut lui accorder des privilèges. Le tableau suivant présente les privilèges système généralement accordés aux développeurs d'applications.

Syntaxe:

GRANT privilege [privilege ...] **To** user [user | role | PUBLIC ...];

- **Privilege:** est le privilège système qui sera assigné ;
- **user | role | PUBLIC:** est le nom de l'utilisateur, le nom du rôle ou PUBLIC qui indique que tous les utilisateurs auront le privilège.

Exemple:

GRANT create session , create table , create sequence , create view **To Asimi;**

Grant succeeded

Privilège système	Operations autorisées
CREATE SESSION	se connecter à la base de données
CREATE TABLE	créer des tables dans le schéma de l'utilisateur
CREATE SEQUENCE	créer des séquences dans le schéma de l'utilisateur
CREATE VIEW	créer des vues dans le schéma de l'utilisateur
CREATE PROCEDURE	créer des procédures stockées, des fonctions ou des packages dans le schéma de l'utilisateur

Généralités et Concepts de Base

Privilèges Objets

Un **privilège objet** est un privilège ou un droit permettant de réaliser une action particulière sur **une table, une vue, une séquence ou une procédure** spécifique. Chaque objet a un jeu particulier des privilèges pouvant être accordés:

GRANT **object-priv** [(columns)] | **ALL ON object** To [user | role | **PUBLIC ...**] ; [WITH GRANT OPTION] ;

- ❖ **object-priv** est un privilège objet ;
- ❖ **ALL** spécifie que tous les privilèges objet ;
- ❖ **columns** indique les colonnes de la table auxquelles les privilèges s'appliquent ;
- ❖ **ON object** est l'objet sur lequel les privilèges s'appliquent ;
- ❖ **TO** identifie à qui les privilèges ont assignés ;
- ❖ **PUBLIC** assigne les privilèges objets à tous les utilisateurs ;
- ❖ **WITH GRANT OPTION** permet à l'utilisateur de donner les privilèges objets à d'autres utilisateurs et rôles.

Exemples:

- ❖ **GRANT select ON INFOR TO Asimi;**
- ❖ **GRANT select, insert ON INFOR TO Asimi WITH GRANT OPTION;**

Privilège objet	Table	Vue	Séquence	Procédure
ALTER	✓		✓	
DELETE	✓	✓		
EXECUTE				✓
INDEX	✓			
INSERT	✓	✓		
REFERENCES	✓			
SELECT	✓	✓	✓	
UPDATE	✓	✓		

Organisation d'une base de données

La **conception** d'une **base de données**, commence par la construction d'un **schéma conceptuel de données**. Ce schéma est constitué **d'entités** et **d'associations** entre ces entités. Il se transforme en **schéma** de base de données en utilisant des **règles** dépendant du **modèle** de la base de données utilisé.

Les données d'une base de données sont organisées sous la forme d'une ou plusieurs tables:

- ❖ Une **table** contient une **collection** de lignes stockées sur le disque de l'ordinateur;
- ❖ Les **tables** doivent respecter les trois **contraintes d'intégrité**;
- ❖ La table devrait avoir un **identifiant** qui désigne toutes les lignes de la Table;
- ❖ **L'intégrité référentielle** est assuré par des clés étrangères entre les tables en relation.

Organisation d'une base de données

Tables, lignes et colonnes

CLIENT					
NCLI	NOM	ADRESSE	LOCALITE	(CAT)	COMPTE
B062	GOFFIN	72, r. de la Gare	Namur	B2	-3200
B112	HANSENNE	23, r. Dumont	Poitiers	C1	1250
B332	MONTI	112, r. Neuve	Genève	B2	0
B512	GILLET	14, r. de l'Eté	Toulouse	B1	-8700
C003	AVRON	8, r. de la Cure	Toulouse	B1	-1700
C123	MERCIER	25, r. Lemaître	Namur	C1	-2300
C400	FERARD	65, r. du Tertre	Poitiers	B2	350
D063	MERCIER	201, bvd du Nord	Toulouse		-2250
F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1	0
F011	PONCELET	17, Clos des Erables	Toulouse	B2	0
F400	JACOB	78, ch. du Moulin	Bruxelles	C2	0
K111	VANBIST	180, r. Florimont	Lille	B1	720
K729	NEUMAN	40, r. Bransart	Toulouse		0
L422	FRANCK	60, r. de Wépion	Namur	C1	0
S127	VANDERKA	3, av. des Roses	Namur	C1	-4580
S712	GUILLAUME	14a, ch. des Roses	Paris	B1	0

schéma

ligne

données

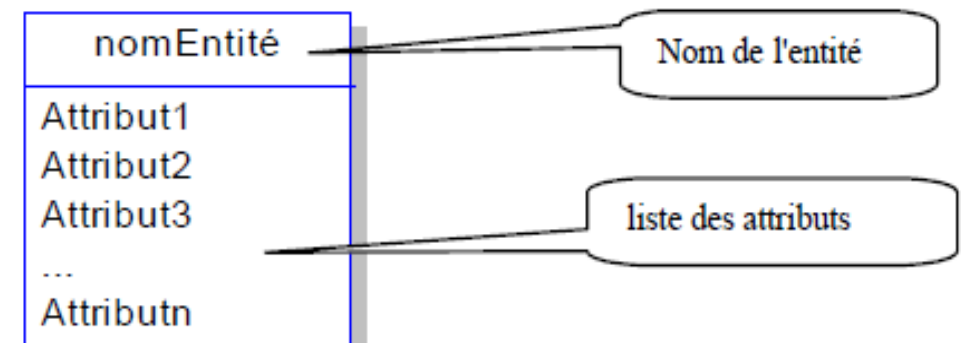
colonne facultative

colonne obligatoire

Entité (Individu ou Objet ou Classe)

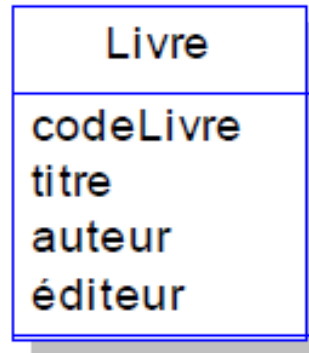
Elle représente, dans le système d'information, un objet **matériel ou immatériel** de l'univers extérieur pour **définir une structure de données**:

- ❖ Elle possède un **nom et des attributs** qui la décrivent dans le contexte de ce sujet.
- ❖ Elle représente **une chose** qui existe et qui peut être distinguée de façon **unique**:
 - ❖ *Ex.: un étudiant, une voiture, une banque;*
- ❖ **Abstraite ou bien concrète ;**
- ❖ Une entité est représentée par le symbole graphique suivant:



Entité (Individu ou Objet ou Classe)

❖ **Exemple:** l'objet **Livre** est une entité dont les attributs sont : **Codelivre**, **Titre**, **Auteur** et **éditeur**.

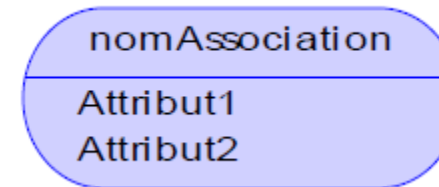
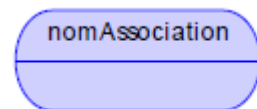


❖ **Remarque:** Les entités d'une base de données ne sont pas **isolées**. Elles sont liées. Cette liaison se fait **par une association**.

Association

Elle représente un **lien entre 2 ou plusieurs entités**. Elle possède un nom et peut avoir à l'image de l'entité **des attributs** la décrivant dans le contexte de la base de données.

- ❖ Une association est représentée par un hexagone:



- **Exemples d'associations:**

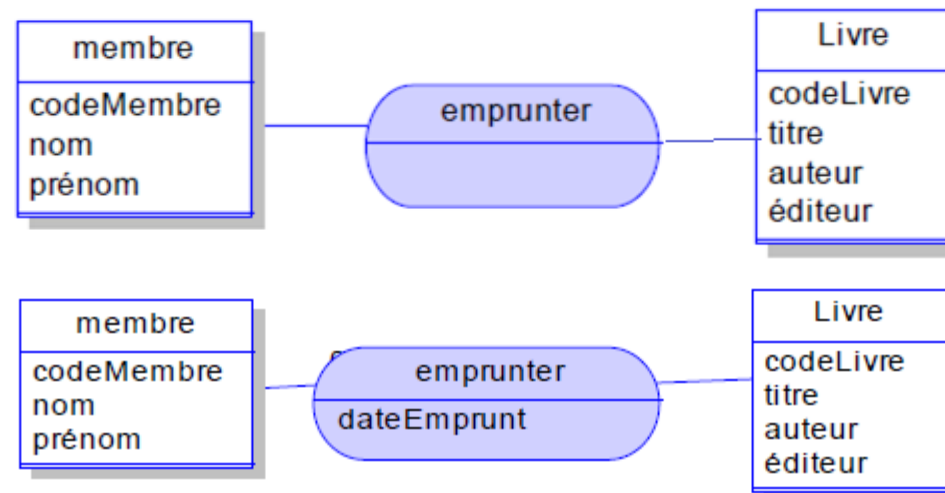
- ❖ En plus de l'entité **Livre**, on considère l'entité **Membre**. Cette entité est décrite par les attributs suivants :
 - ❖ (Numéro du membre, nom, prénom, email, date d'inscription)

Association

Les entités Membre et Livre sont liées par l'association *Emprunter* qui modélise l'information suivante :

❖ *des membres empruntent des livres.*

Le schéma de cette association est :



Association

- ❖ L'association "**Emprunter**" lie deux entités membre et livre. On dit que c'est une **association binaire ou de degré 2-aires**.
- ❖ Une association peut aussi lier trois entités : **association ternaire ou 3-aire**.
- ❖ On trouve aussi des associations qui lient une entité à elle-même : **association réflexive ou récursive**.
 - ❖ Un dossier contient d'autres dossiers.
 - ❖ Une personne se marie avec une autre personne.
 - ❖ Un employé est chef d'autres employés.
- ❖ En général une association peut **lier deux ou plusieurs entités ou une entité avec elle-même**.
- ❖ **Remarque: Les entités sont souvent nommées par des noms, alors que les associations sont nommées par des verbes.**

Occurrences

Les occurrences comportent trois classes :

- ❖ L'occurrence d'une **Entité**.
- ❖ L'occurrence d'une **Propriété**.
- ❖ L'occurrence d'une **Association**.

Définition 1 : L'occurrence d'une propriété est l'une des valeurs qu'elle peut prendre.

Exemple : La propriété prénom de l'étudiant peut avoir comme occurrence **ali, brahim,**

Définition 2 : L'occurrence d'une entité est un ensemble formé par une occurrence de chacune de ses propriétés.

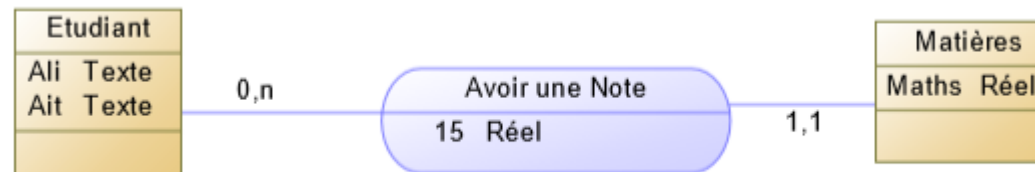
Occurrences

Une **occurrence** de **l'entité** étudiant est :

Etudiant
kalila ali 31, rue essalam, agadir

Définition 3 : L'occurrence d'une Association est une occurrence des entités associées. C'est donc l'occurrence de chacune des propriétés qu'elle porte correspondant aux occurrences d'entités associées.

Exemple :



Identifiant d'une entité

Un **identifiant** d'une entité est l'une de ses propriétés permettant **d'identifier** de **manière unique** (**clef primaire** dans BD) une ligne dans une table.

- ❖ Un identifiant peut être **composé** (une ou plusieurs colonnes);
- ❖ Une table peut posséder **plusieurs identifiants**. On choisit l'un d'eux, qu'on déclare **primaire** (**clé primaire**). Les autres sont dès lors **secondaires**.
- ❖ Toute entité doit avoir un **identifiant**, sinon il faut renoncer à la conception d'une telle entité.
- ❖ **L'occurrence de l'identifiant est suffisante** pour définir l'occurrence de l'entité.
- ❖ Un identifiant d'une association est la **concaténation** des identifiants des entités associées;
- ❖ Il est possible de déclarer une **table sans identifiant** mais ceci n'est **pas recommandé**.

Cardinalité

La cardinalité ou bien la multiplicité d'une entité est le couple formé d'une borne minimale et d'une borne maximale de participations de cette entité à une telle association, notée par : **min.max**.

- ❖ **La borne minimale (min)** décrit le nombre minimum de fois **d'occurrences** d'une entité qui peuvent participer à une relation.
- ❖ **La borne maximale (max)** décrit le nombre maximum de fois **d'occurrences** d'une entité qui peuvent participer à une relation.
- ❖ Une cardinalité (1,N) signifie que chaque occurrence d'entité participe à une relation :
 - ❖ Au moins une fois si N variable.
 - ❖ Au moins une fois et au plus N fois si N est fixé.

Cardinalité

- ❖ Une cardinalité (1,1) signifie que chaque **occurrence** d'entité participe une et une seule fois à une relation.
- ❖ Une cardinalité (0,N) signifie qu'il existe au moins une occurrence d'entité qui ne participe pas forcément à la relation et d'autre :
 - ❖ Plusieurs fois si N variable.
 - ❖ Au plus N fois si N est fixé.
- ❖ **Exemple** : Considérons les règles de gestion suivantes :
 - ❖ RG1 : Un étudiant appartient à une seule classe.
 - ❖ RG2 : Une classe contient un ou plusieurs étudiants.



Cardinalité

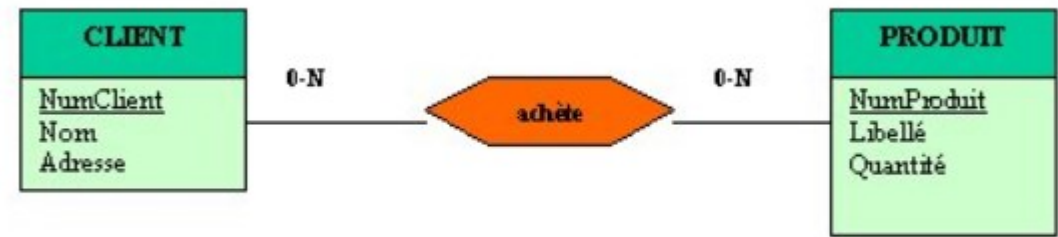
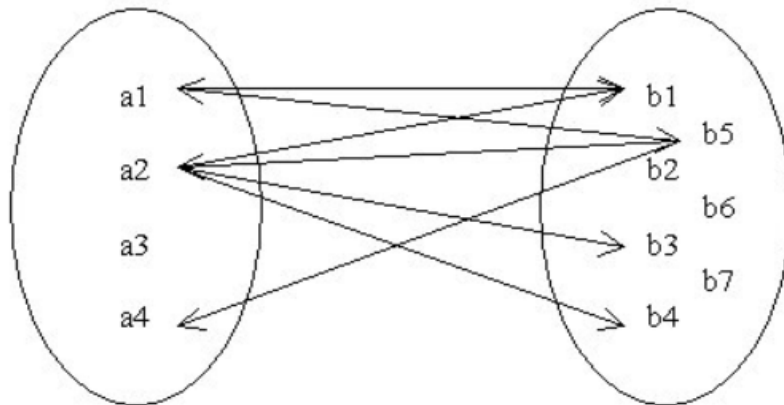
- ❖ La cardinalité (1,1) signifie qu'un étudiant ne peut appartenir qu'à une seule classe.
- ❖ La cardinalité (1,N) signifie qu'une classe peut contenir au minimum un étudiant et au maximum un nombre inconnu d'étudiants.

Cardinalité

Une cardinalité (0,N) signifie qu'il existe au **moins** une **occurrence** d'entité qui ne participe pas forcément à la relation et d'autre :

Ici, une entité de chaque ensemble peut être connectée à plusieurs entités de l'autre ensemble:

- ❖ Plusieurs fois si N variable.
- ❖ Au plus N fois si N est fixé.

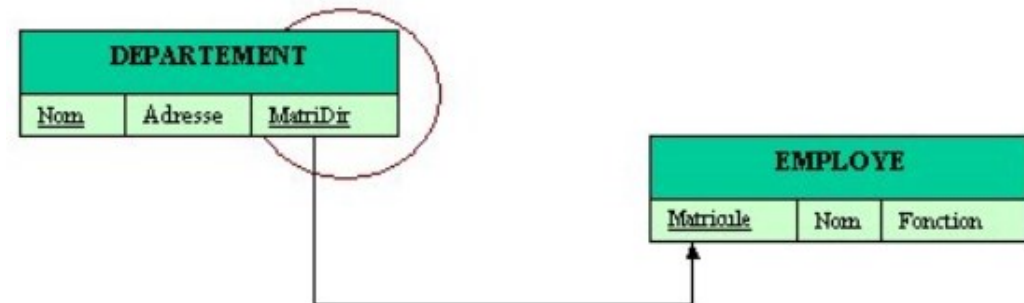
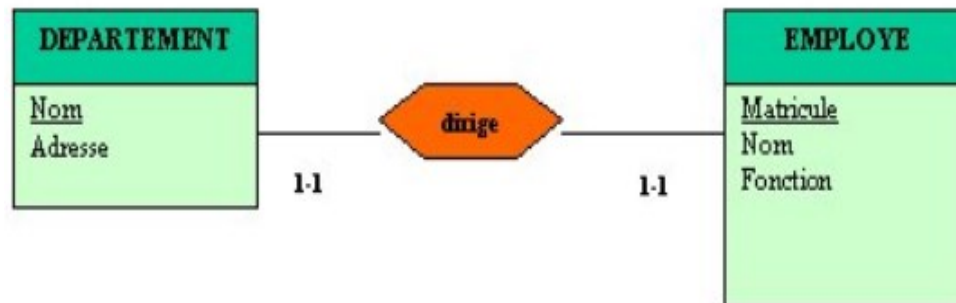
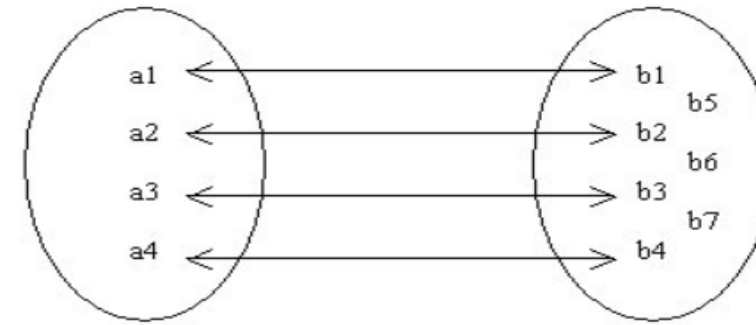


Cardinalité

Une cardinalité (1,1) signifie que chaque **occurrence** d'entité participe **une** et **une** seule fois à une relation:

Exemple:

- Un département est dirigé par un employé;
- Un employé appartient à un seul département.

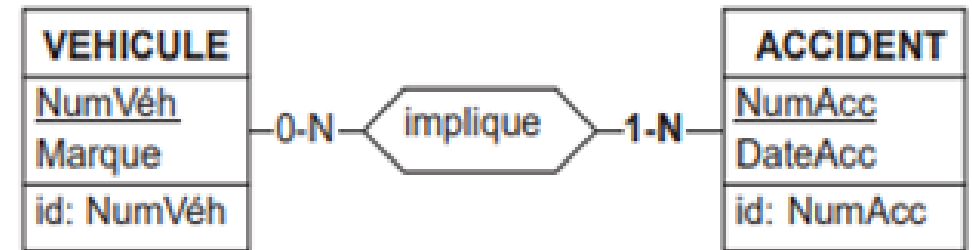
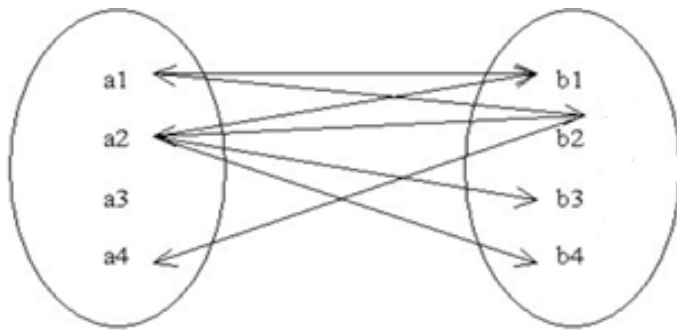


La plus imposée (département) prend la cardinalité 1,1.

Cardinalité

Une cardinalité (1,N) signifie que chaque **occurrence** d'entité participe à une relation :

- ❖ Au moins une fois si N variable.
- ❖ Au moins une fois et au plus N fois si N est fixé.





LANGAGE SQL DDL

Le langage LDD comprend les commandes **Create**, **Alter** et **Drop**:

- ❖ La commande **Create** est pour créer des bases de données et les objets de base de données tels que les tables, les vues, les utilisateurs, etc.
- ❖ La commande **Alter** sert à modifier la structure des objets de base de données.
- ❖ La commande **Drop** supprime les objets de base de données.

LANGAGE SQL DDL

La commande **Create** sert à créer les objets de bases de données :

- ❖ Bases de données
- ❖ Tables
- ❖ Index
- ❖ Vues
- ❖ rôles
- ❖ Séquences
- ❖ Etc

LANGAGE SQL DDL

Connexion sous Oracle

- ❖ Connect
- ❖ Enter user-name: system
- ❖ Enter password:*****
- ❖ Connected

LANGAGE SQL DDL

Création d'une Base de données

Pour créer une base de données, on utilise la syntaxe suivante (dépend de la version d'Oracle) :

❖ Syntaxe:

❖ **Create database nom_base ;**

❖ Nom_base est un identificateur limité à 8 caractères.

❖ Exemple:

❖ **Create database EstClient ;**

LANGAGE SQL DDL

Création d'une Table (CREATE TABLE)

Pour pouvoir créer une table dans un schéma, l'utilisateur doit avoir le privilège **CREATE TABLE** ou bien le privilège **CREATE ANY TABLE**.

La syntaxe SQL :

- ❖ **CREATE TABLE** [*schéma.*]*nomTable*
- ❖ (*colonne1 type1* [DEFAULT *valeur1*] [NOT NULL]
- ❖ [, *colonne2 type2* [DEFAULT *valeur2*] [NOT NULL]]
- ❖
- ❖ [CONSTRAINT *nomContrainte1 typeContrainte1*]...) ;

Remarque: la valeur **NULL** représente une valeur non disponible, non affectée, inconnue.... Elle est totalement différente:

- ❖ d'un espace pour un caractère;
- ❖ d'un zéro pour un nombre.

LANGAGE SQL DDL

Schéma d'une Table (CREATE TABLE)

Le schéma d'une table définit sa structure:

- ❖ Nom de la **table**,
- ❖ Pour chaque **colonne**, son nom, son domaine, son caractère obligatoire;
- ❖ **Identifiant** de la table(liste des colonnes);
- ❖ **Identifiants** secondaires éventuels (liste des colonnes);
- ❖ **Clés étrangères** éventuelles (liste des colonnes et table cible).

LANGAGE SQL DDL

Tables, lignes et colonnes

CLIENT					
NCLI	NOM	ADRESSE	LOCALITE	(CAT)	COMPTE
B062	GOFFIN	72, r. de la Gare	Namur	B2	-3200
B112	HANSENNE	23, r. Dumont	Poitiers	C1	1250
B332	MONTI	112, r. Neuve	Genève	B2	0
B512	GILLET	14, r. de l'Eté	Toulouse	B1	-8700
C003	AVRON	8, r. de la Cure	Toulouse	B1	-1700
C123	MERCIER	25, r. Lemaître	Namur	C1	-2300
C400	FERARD	65, r. du Tertre	Poitiers	B2	350
D063	MERCIER	201, bvd du Nord	Toulouse		-2250
F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1	0
F011	PONCELET	17, Clos des Erables	Toulouse	B2	0
F400	JACOB	78, ch. du Moulin	Bruxelles	C2	0
K111	VANBIST	180, r. Florimont	Lille	B1	720
K729	NEUMAN	40, r. Bransart	Toulouse		0
L422	FRANCK	60, r. de Wépion	Namur	C1	0
S127	VANDERKA	3, av. des Roses	Namur	C1	-4580
S712	GUILLAUME	14a, ch. des Roses	Paris	B1	0

schéma

ligne

données

colonne facultative

colonne obligatoire

LANGAGE SQL DDL

Création d'une Table

Cette opération produit une table vide. On spécifie le nom de la table et la description de ses colonnes. Également, on spécifiera pour chaque colonne son nom et son domaine:

```
Create Table CLIENT( NCLI char(10),  
                     NOM char(32),  
                     ADRESSE char(60),  
                     LOCALITE char(30),  
                     CAT char(2),  
                     COMPTE decimal(9,2));
```

LANGUAGE SQL DDL

Les colonnes admet divers domaines de données dans la déclaration d'une colonne d'une table:

- ❖ **INTEGER (ou INT)** : entiers signés longs (p. ex. 32 bits);
- ❖ **NUMERIC(p,q)** : nombres décimaux de p chiffres dont q après le point décimal; si elle n'est pas mentionnée, la valeur de q est 0;
- ❖ **DECIMAL(p,q)** : nombres décimaux d'au moins p chiffres dont q après le point décimal; si elle n'est pas mentionnée, la valeur de q est 0;
- ❖ **FLOAT(p)** : nombres en virgule flottante d'au moins p bits significatifs;
- ❖ **CHARACTER(p) (ou CHAR)** : chaîne de longueur fixe de p caractères;
- ❖ **VARCHAR(p)** : chaîne de longueur variable d'au plus p caractères;
- ❖ **DATE** : dates (année, mois et jour);
- ❖ **TIME** : instants (heure, minute, seconde, éventuellement 1000^{ème} de seconde);
- ❖ **TIMESTAMP** : date + temps,

LANGAGE SQL DDL

Contraintes d'intégrité

Elles permettent la **simplification** du code des applications et **renforcement de la sécurité** par la vérification automatique de ces contraintes, plutôt, il exigent un langage de **définition de contraintes d'intégrité**:

- ❖ Définir un état **cohérent** de la base de données;
- ❖ Vérifier **automatiquement** à chaque **insertion**, **modification** ou **suppression** les données.

Il existe trois contraintes d'intégrité :

- ❖ **Contrainte de Domaine:** Les valeurs d'un attribut doivent appartenir à son domaine;

```
CLIENT (  Num_client : entier long ,  
          Nom : chaîne de 30 caractères,  
          Prenom : chaîne de 20 caractères,  
          Telephone : chaîne de 10 chiffres );
```

LANGAGE SQL DDL

Contraintes d'intégrité

- ❖ **Contrainte de Relation:** Chaque relation possède **une clé primaire unique et non nulle**. La clé primaire est soulignée dans un schéma de relation:

```
CLIENT (    Num_client : entier long ,  
            Nom : chaîne de 30 caractères,  
            Prenom : chaîne de 20 caractères,  
            Telephone : chaîne de 10 chiffres );
```

- ❖ **Contrainte de Référence:** La valeur d'une **clé étrangère** dans la relation référençant doit figurer comme valeur de la **clé primaire associée** dans la relation référenciée:

- ETUDIANTS (NumEtud, NomEtud, PrenEtud, AdrEtud, VilleEtud),
- MATIERES (NumMat, NomMet, Coef, Intitule),
- NOTES (# NumEtud, # NumMat, Date, Note).

LANGAGE SQL DDL

Contraintes

Les contraintes ont pour but d'automatiser les règles de gestion au niveau des colonnes des tables. Elles peuvent être déclarées:

- ❖ **En ligne:** même temps que la colonne ;
- ❖ **Contrainte nommée:** n'est pas limitée à une colonne et peut être personnalisée par un nom. Pour nommer une contrainte, on doit utiliser la clause **CONSTRAINT**.

Oracle recommande de déclarer les contraintes **NOT NULL** en ligne, les autres peuvent être déclarées soit en ligne, soit nommées. Les syntaxes pour créer les contraintes sont:

CONSTRAINT *nomContrainte*

- ❖ **UNIQUE** (*colonne1* [,*colonne2*]...)
- ❖ **PRIMARY KEY** (*colonne1* [,*colonne2*]...)
- ❖ **FOREIGN KEY** (*colonne1* [,*colonne2*]...) **REFERENCES** [*schéma.*]*nomTablePere* (*colonne1* [,*colonne2*]...) [**ON DELETE** { **CASCADE** | **SET NULL** }]
- ❖ **CHECK** (*condition*)

LANGUAGE SQL DDL

Conventions recommandées

Pour nommer les contraintes, on doit suivre les conventions recommandées suivantes:

- ❖ Préfixer **par pk_** le nom d'une contrainte **clé primaire**, **fk_** une **clé étrangère**, **ck_** une **vérification**, **un_** une **unicité**.
- ❖ Suffixer le nom d'une contrainte clé primaire par nom de la table (**pk_Client**).
- ❖ Suffixer le nom d'une contrainte **clé étrangère** par les noms de la table source, de la clé étrangère, et de la table cible (**fk_Commande_NCli_Client**).

❖ Exemple:

- ❖ CREATE TABLE Compagnie
 - (comp CHAR(4), nrue NUMBER(3),.....
 - nomComp CHAR(15) NOT NULL,
 - CONSTRAINT **pk_Compagnie** PRIMARY KEY(comp));

LANGAGE SQL DDL

Clause Unique

La contrainte **UNIQUE** impose une valeur distincte au niveau de la table. Cette clause exprime **une clé candidate** dans une table. Cette clé peut être utilisé comme **une clé étrangère pour assurer la contrainte référentielle entre deux tables**.

- ❖ Le champ est unique. Donc, il n'accepte pas des valeurs identiques.

La syntaxe est:

❖ CREATE TABLE Pilote

- (brevet CHAR(6), nom CHAR(15),...,
- **CONSTRAINT un_nom UNIQUE (nom),**
- **CONSTRAINT pk_Pilote PRIMARY KEY(brevet),**
-

LANGAGE SQL DDL

Valeur par défaut

Le mot-clé **DEFAULT** sert à affecter explicitement une valeur par défaut à une colonne donnée d'une table. Ainsi, on peut assigner automatiquement à des colonnes des valeurs par défaut:

❖ Syntaxe: **Default expression**

❖ Exemple 1:

❖ ... **CAT char(2) default 'AA'**

❖ Exemple 2:

- ❖ CREATE TABLE Compagnie
- ❖ (comp CHAR(4), nrue NUMBER(3),
- ❖ **rue CHAR(20), ville CHAR(15) DEFAULT 'Paris',**
- ❖ nomComp CHAR(15) **NOT NULL,**
- ❖ **CONSTRAINT pk_Compagnie PRIMARY KEY(comp));**

LANGAGE SQL DDL

Contrainte Clé primaire

Dans une même table, nous pouvons trois clés: une contrainte de clé primaire (**Primary key**), une contrainte de clé candidate (**Unique**) et une clé étrangère (**Foreign key**). L'une est **primaire** plutôt les autres sont des clés **secondaires**.

Une table exige **un identifiant minimal** garantir la **contrainte de relation**. Cette clé primaire définit par la clause **Primary key**:

- ❖ L'ordre lors de la création des contraintes nommées (**PK**) n'est pas important.
- ❖ Il existe au moins une colonne qui sert à identifier toutes les lignes: **c'est la clé primaire;**
- ❖ La clause **PRIMARY KEY** est équivalent à :

UNIQUE + NOT NULL + index.

- ❖ L'ordre de création des tables est important quand on définit les contraintes en même temps que les tables

LANGAGE SQL DDL

Contrainte Clé primaire

Une table peut posséder **plusieurs identifiants**. On choisit l'un d'eux, qu'on déclare **primaire**. Les autres sont des clés **secondaires**. La clause **Primary key** sert à spécifier la clé primaire d'une table:

❖ Exemples:

- ❖ create table CLIENT (NCLI char(10) **primary key** ,
NOM char(32), ADRESSE char(60),,
COMPTE decimal (9,2));
//primary key (NCLI)
- ❖ create table DETAIL (NCOM char(12), NPRO char(15),
QCOM decimal(8),
primary key (NCOM,NPRO))

Rq: Un **identifiant** définit une **contrainte d'unicité et d'obligation**.

LANGAGE SQL DDL

Contrainte Clé étrangère

Une **clé étrangère** définit un lien de connexion entre tables: une table « père » et table « fils ou bien enfant ». Il traduit l'intégrité **référentielle entre les tables**. Cette contrainte exige un **niveau de cohérence entre les tables en relation très élevé**:

- ❖ **L'ordre** de création des clés très est important;
- ❖ Les clés étrangères peuvent être nulles si aucune contrainte **NOT NULL** n'est déclarée.
- ❖ Les clauses **Foreign key et References** servent pour déclarer une clé étrangère dans une table cible:
- ❖ Une clé étrangère peut référencer une **clé primaire ou une clé candidate de la table « père »**.
- ❖ Nous pouvons activer ou bien désactiver les clés.

LANGAGE SQL DDL

Clé étrangère

Identifiant

CLIENT					
NCLI	NOM	ADRESSE	LOCALITE	(CAT)	COMPTE
B062	GOFFIN	72, r. de la Gare	Namur	B2	-3200
B112	HANSENNE	23, r. Dumont	Poitiers	C1	1250
B332	MONTI	112, r. Neuve	Genève	B2	0
B512	GILLET	14, r. de l'Été	Toulouse	B1	-8700
C003	AVRON	8, r. de la Cure	Toulouse	B1	-1700
C123	MERCIER	25, r. Lemaître	Namur	C1	-2300
C400	FERARD	65, r. du Tertre	Poitiers	B2	350
D063	MERCIER	201, bvd du Nord	Toulouse		-2250
F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1	0
F011	PONCELET	17, Clos des Erables	Toulouse	B2	0
F400	JACOB	78, ch. du Moulin	Bruxelles	C2	0
K111	VANBIST	180, r. Florimont	Lille	B1	720
K729	NEUMAN	40, r. Bransart	Toulouse		0
L422	FRANCK	60, r. de Wépion	Namur	C1	0
S127	VANDERKA	3, av. des Roses	Namur	C1	-4580
S712	GUILLAUME	14a, ch. des Roses	Paris	B1	0

COMMANDE		
NCOM	NCLI	DATECOM
30178	K111	21/12/2008
30179	C400	22/12/2008
30182	S127	23/12/2008
30184	C400	23/12/2008
30185	F011	2/01/2009
30186	C400	2/01/2009
30188	B512	3/01/2009

Clé étrangère

LANGAGE SQL DDL

Contrainte Clé étrangère

Une **clé étrangère** définit une **contrainte référentielle**. Les clauses **Foreign key** et **References** servent pour déclarer une clé étrangère dans une table référencée:

❖ **Exemple:**

```
Create Table DETAIL ( NCOM char(12),  
                    NPRO char(15),  
                    QCOM decimal(8),  
                    primary key (NCOM,NPRO),  
                    Foreign key (NCOM) references COMMANDE,  
                    Foreign key (NPRO) references PRODUIT (colonne,...) ) ;
```

Rq: Une clé étrangère référence en principe l'**identifiant primaire** de la table cible.

LANGAGE SQL DDL

Conventions et Remarques

- ❖ La valeur **NULL** est interdite pour une clé primaire;
- ❖ On souligne les clés primaires;
- ❖ On fait précéder d'un dièse # les clés étrangères;
 - Clients(numclient, nom client, prénom, adresse);
 - Commandes(numcommande, date commande, #numclient);
- ❖ Une même table peut avoir une ou plusieurs clés étrangères mais une seule clé primaire;
- ❖ Une clé étrangère peut être composée (Si la clé primaire père est composée);
- ❖ Une clé étrangère ne doit pas recevoir la valeur **Null**,

LANGAGE SQL DDL

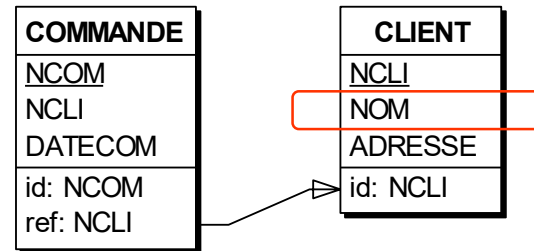
Modifications et contraintes d'intégrité

La modification des valeurs d'une colonne donnée ne sera effectuée que si son résultat ne viole **aucune contrainte d'intégrité**.

- ❖ On distingue trois contraintes d'intégrité :
 - Colonne obligatoire;
 - Contrainte d'unicité;
 - Contrainte référentielle.
- ❖ Nous traitons trois opérations élémentaires de modification sur une colonne donnée:
 - Insérer une ligne;
 - Supprimer une ligne;
 - Modifier une valeur de colonne d'une ligne.

LANGAGE SQL DDL

Colonne obligatoire:



Colonne NOM obligatoire

Insérer une ligne
de CLIENT

La valeur de NOM doit être *not null*

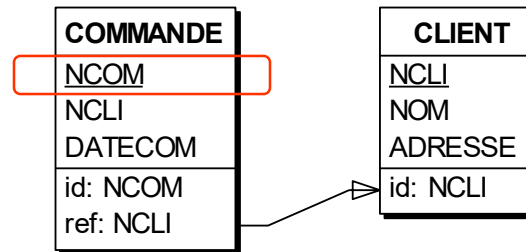
supprimer une ligne
de CLIENT

Modifier la valeur de NOM
de CLIENT

La nouvelle valeur de NOM doit être *not null*

LANGAGE SQL DDL

Identifiant:



{NCOM} identifiant de COMMANDE

**Insérer une ligne
de COMMANDE**

**La valeur de NCOM ne doit pas être déjà présente
dans une ligne de COMMANDE**

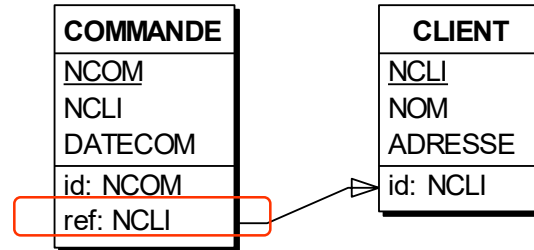
**Supprimer une ligne
de COMMANDE**

**Modifier la valeur de
NCOM de
COMMANDE**

**La nouvelle valeur de NCOM ne doit pas être déjà
présente dans une ligne de COMMANDE**

LANGAGE SQL DDL

Clé étrangère:



{NCLI} clé étrangère de COMMANDE vers CLIENT

Insérer une ligne de COMMANDE

La valeur de NCLI doit être présente dans la colonne NCLI d'une ligne de CLIENT

Supprimer une ligne de COMMANDE

Modifier la valeur de NCLI de COMMANDE

La nouvelle valeur de NCLI doit être présente dans la colonne NCLI d'une ligne de CLIENT

Supprimer une ligne de CLIENT

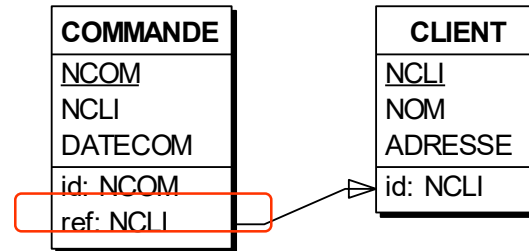
L'intégrité référentielle doit être satisfaite après l'opération

Modifier la valeur de NCLI de CLIENT

L'intégrité référentielle doit être satisfaite après l'opération

LANGUAGE SQL DDL

Clé étrangère:

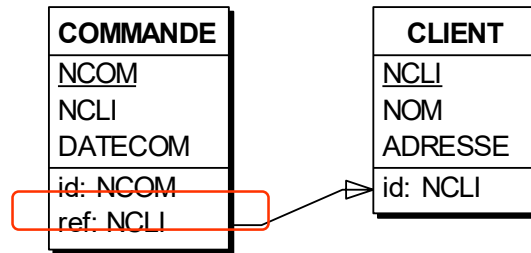


Supprimer une ligne de CLIENT

Comportement	{NCLI} clé étrangère de COMMANDE vers CLIENT
1. mode <i>no action</i>	Opération refusée s'il existe des lignes de COMMANDE dépendantes
2. mode <i>cascade</i>	Ligne supprimée mais aussi les lignes de COMMANDE dépendantes
3. mode <i>set null</i>	Si NCLI de COMMANDE facultative, la colonne NCLI des lignes dépendantes de COMMANDE est mise à <i>null</i>
4. mode <i>set default</i>	(<i>default</i> pour NCLI de COMMANDE) la colonne NCLI des lignes dépendantes de COMMANDE est mise à la <i>valeur par défaut</i>

LANGAGE SQL DDL

Clé étrangère



Modifier la valeur
de NCLI de CLIENT

Comportement

{NCLI} clé étrangère de COMMANDE vers CLIENT

1. mode *no action*

Opération **refusée** s'il existe des lignes de COMMANDE dépendantes;

2. mode *cascade*

Opération réalisée + valeurs de NCLI adaptées dans les lignes de COMMANDE dépendantes;

3. mode *set null*

(si NCLI de COMMANDE facultative) opération réalisée + NCLI mis à *null* dans les lignes de COMMANDE dépendantes;

4. mode *set default*

(*default* pour NCLI de COMMANDE) opération réalisée + NCLI mis à *default* dans les lignes de COMMANDE dépendantes;

LANGUAGE SQL DDL

Colonne obligatoire/facultative

Par défaut toute colonne est facultative. La clause **not null** nous permet de déclarer **une colonne obligatoire**:

❖ Exemple:

❖ Create Table OFFRE (NUMFL char(10) **not null**,
NUMPL char(15) **not null**,
PRIX decimal(8),
primary key (NUMFL, NUMPL),
Foreign key (NUMFL) references FOURNISSEUR,
Foreign key (NUMPL) references PIECE);

LANGAGE SQL DDL

Application 1: Création des tables sous Oracle

- ❖ Nous considérons les tables suivantes:
 - ❖ Usine(NumUsine, NomUsine, ville);
 - ❖ Produit(NumProduit, NomProduit, CouleurProduit, PoidProduit)
 - ❖ Fournisseur(NumFour, NomFour, VilleFour);
- ❖ Une association ternaire entre Usine, Produit et Fournisseur;
 - ❖ PUF(# NumFour, # NumUsine, # NumProduit, Quantite)

❖ Questions:

- ❖ Construire la table Usine en précisant que la colonne NomUsine est obligatoire;
- ❖ Construire la table Produit en précisant que les colonnes NomProduit et PoidProduit sont obligatoires;
- ❖ Construire la table Fournisseur en précisant que la colonne NomFour est facultative;
- ❖ Construire la table PUF en précisant que la colonne Quantite est obligatoire;

DUT-Informatique –S3

2021/2022

LANGAGE SQL DDL

❖ La modification d'une table comprend :

- ❖ Ajout des colonnes;
- ❖ Ajout des contraintes ;
- ❖ Modification des colonnes;
- ❖ Suppression des colonnes;
- ❖ Suppression des contraintes ;
- ❖ Activation/Désactivation des contraintes ;
- ❖ Etc.

LANGAGE SQL DDL

Rôles des commandes

La commande **Rename to** sert à **changer (renommer)** le nom:

❖ **Table , Colonne et Contrainte.**

- ❖ Le mot **add** est utilisé pour ajouter une colonne;
- ❖ Les mots **add constraint** sont utilisés pour ajouter une contrainte.
- ❖ Le mot **drop** supprime une table, une colonne, une contrainte On peut l'utiliser **avec l'option cascade** pour supprimer les contraintes qui font référence à l'attribut ou avec **cascade constraints** pour supprimer les contraintes basées sur la contrainte à supprimer.

LANGAGE SQL DDL

Rôles des commandes

Le mot **modify** sert à modifier les colonnes ou les contraintes. Cette modification comprend: **le type, la valeur par défaut et les contraintes définies sur la colonne.**

❖ |**Modify Colonne** [**type**] [**default valeurParDefaut**] [**contrainteColonne**]

❖ Les mots **modify constraint** permettent la modification d'une contrainte qui **consiste à l'activer ou à la désactiver:**

❖ Le mot **enable** active une contrainte;

❖ Le mot **disable** désactive une contrainte;

❖ |**Modify constraint** nomContrainte **enable|disable** [cascade] .

LANGAGE SQL DDL

Rôles des commandes

La syntaxe simplifiée de modification de table est :

❖ Alter table nomTable

- ❖ |**rename to** nouveauNomDeTable;
- ❖ |**rename column** Colonne **to** nouveauNomColonne;
- ❖ |**rename constraint** contrainte **to** nouveauNom;
- ❖ |**add champ type** [**default** valParDéfaut] [contrainte_colonne] ;
- ❖ |**add(champ1**type...|contrainteTable1,**champ2** type...|contrainteTable2...);

LANGUAGE SQL DDL

Rôles des commandes

- ❖ **drop column** champ [cascade constraints] ;
- ❖ **drop** (champ1, champ2, ...) [cascade constraints]
- ❖ **drop constraint** nomContrainte [cascade]
- ❖ **drop primary key** [cascade]
- ❖ **drop unique(champ1[,champ2...])** [cascade]
- ❖

LANGUAGE SQL DDL

Renommer une Colonne/Table

Pour renommer une colonne d'une table, nous utilisons :

- ❖ **ALTER TABLE** *table_name* **RENAME COLUMN** *old_name* **TO** *new_name*;

- ❖ **ALTER TABLE** *table_name* **CHANGE COLUMN** *old_name* **TO** *new_name*;

- ❖ Pour changer le nom de la table:

- ❖ **ALTER TABLE** *table_name* **RENAME TO** *new_table_name*;

- ❖ Exemple:

- ❖ **Alter table** *emp* **rename to** *empl*;

LANGAGE SQL DDL

Suppression des tables

Pour supprimer une table et les données qu'elle contient, on utilise la **commande drop**.

❖ Syntaxe:

Drop Table nom_table [cascade constraints] ;

L'option **cascade constraints (contraintes référentielles)** est nécessaire si **la table est référencée par une autre table**. On l'utilise pour supprimer la contrainte utilisant la table qu'on veut supprimer.

❖ Exemples:

❖ **Drop Table service cascade constraints ;**

LANGAGE SQL DDL

Ajout d'une colonne

Il est possible d'ajouter une colonne à une table donnée. La colonne ajoutée prends des **valeurs null** pour toutes les lignes.

- ❖ On ne peut ajouter une **colonne obligatoire** que si la table est **vide**, ou si cette colonne possède une valeur **par défaut**.
- ❖ **La** commande **Alter** permet la modification de la **structure des objets** de base de données.
- ❖ La commande suivante ajoute la colonne POIDS à la table PRODUIT :
 - ❖ **Alter Table PRODUIT add POIDS Integer ;**
 - ❖ **Alter Table usine add (num int, ville varchar2(20));**

LANGAGE SQL DDL

Suppression d'une colonne

Il est possible de retirer une colonne d'une table. Cette opération s'effectue à l'aide d'une commande suivante :

- ❖ **Alter Table PRODUIT Drop Column PRIX;**
- ❖ Suppression de la colonne PRIX de la table PRODUIT;
- ❖ Il est également possible de changer la valeur par défaut d'un domaine:
 - ❖ **Alter Table CLIENT Alter Column CAT set '00' ;**
 - ❖ Nous pouvons aussi utiliser la clause: **(Modify Colonne)**;
- ❖ Ou bien de supprimer définitivement un domaine :
 - ❖ **Drop domain MATRICULE;**

LANGAGE SQL DDL

Ajout d'une contrainte

En générale les contraintes devraient être déclarer lors de la création d'une table. Mais, Il est cependant possible de les ajouter et même de les retirer a posteriori par une commande **Alter Table**.

❖ Ajout d'une clé primaire:

❖ **Alter Table Client Add Constraint cl1 primary key (NCLI);**

❖ **Alter table Individu add Constraint cil foreign key (n_logement) references logement;**

❖ **Attention** , la demande d'ajout d'un **identifiant** sera **refusée** si les **données** que la **table** contient **déjà violent cette propriété** ;

LANGAGE SQL DDL

Colonne obligatoire et clés étrangères

Une **colonne obligatoire** peut être re-déclarée facultative et inversement:

- ❖ **Alter Table CLIENT modify CAT not null;**

- ❖ **Alter Table CLIENT modify ADRESSE null ;**

- ❖ Une clé étrangère peut aussi **être définie ou retirée** après création de la table:

- ❖ **Alter Table Commande Add Constraint name_cont2 foreign key (NCLI) references CLIENT;**

- ❖ **Alter Table Commande Drop Constraint name_cont2;**

LANGUAGE SQL DDL

Renommer une contrainte

Il est aussi possible de nommer une contrainte lors la définition ou dans la modification d'une table donnée. On nomme une contrainte par la clause suivante:

- ❖ **Constraint** nomContrainte....;
- ❖ Nommer une contrainte lors de la création d'une table:
 - ❖ **Create Table** DETAIL (NCOM char(12) **not null**,
NPRO char(15) **Constraint C1 not null**,
QCOM decimal(8),
constraint pk_DETAIL **primary key** (NCOM,NPRO),
constraint fk_Produit_Nprod_DETAIL **foreign key** (NPRO) **references** PRODUIT,
constraint fk_Produit...)

LANGUAGE SQL DDL

Renommer/supprimer une contrainte

❖ Renommer une contrainte d'une table:

❖ **Alter Table** CLIENT **Add constraint** Un_CLI_U **unique** (NOM,ADRESSE,LOCALITE) ;

❖ **Alter Table** COMMANDE **Add constraint** fk_CLIENT_NCLI_COMMANDE **foreign key** (NCLI) **references** CLIENT ;

❖ Il est possible de supprimer une contrainte existante :

❖ **Alter Table** DETAIL **drop constraint** fk_CLIENT_NCLI_COMMANDE;

LANGAGE SQL DDL

❖ Construire la table suivante :

❖ `emp(nemp, (Nom), Prénom, Salaire , Genre);`

❖ Puis expliquer le rôle des commandes suivantes:

- ❖ Nommer les contraintes;
- ❖ Alter table emp disable primary key;
- ❖ Alter table emp add column email varchar2(50);
- ❖ Alter table emp drop email;
- ❖ Alter table emp drop primary key;
- ❖ Alter table emp drop foreign key(ncient);
- ❖ Alter table emp disable unique(nom);

LANGAGE SQL DDL

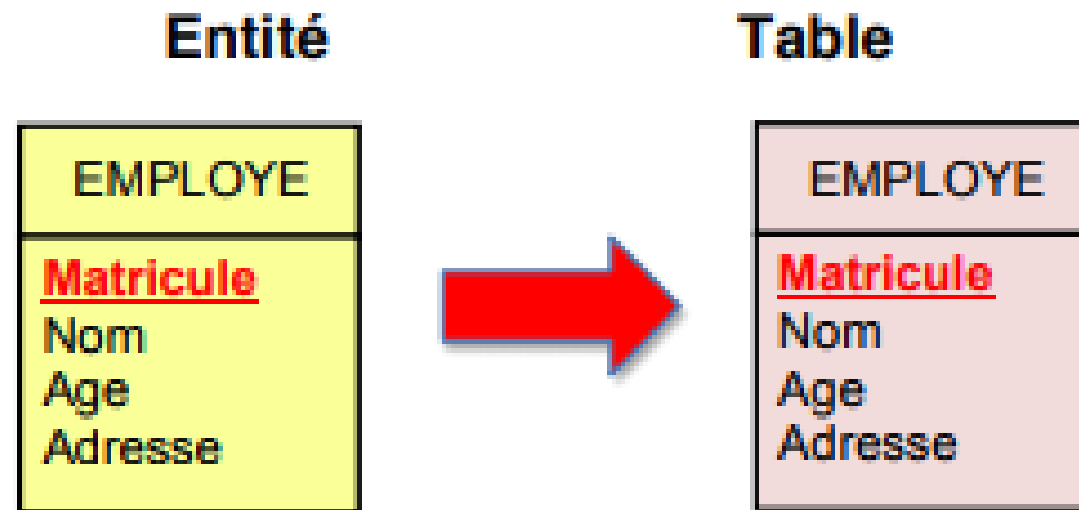
Règles de passage d'un MCD au MLD

Cette traduction s'effectue indépendamment des langages de programmation. Effectivement, elle dépend du **type de la base de données à utiliser pour créer votre application**. Ces règles sont décrites comme suites:

- 1) Toute **entité** devient une **table**;
- 2) Relation binaire: $(0,N) \sim (1,1)$ ou $(1,N) \sim (1,1)$ ou $(0,N) \sim (0,1)$ ou $(1,N) \sim (0,1)$;
 - On **duplique** la clé primaire de la **table 0,N ou 1,N** dans la **table 1,1 ou 0,1**, elle devient une **clé étrangère**;
- 3) Relation ternaire ou supérieure:
 - L'association se traduit par une table supplémentaire dont la clé primaire est **composée d'autant de clés étrangères que d'entité en association**.

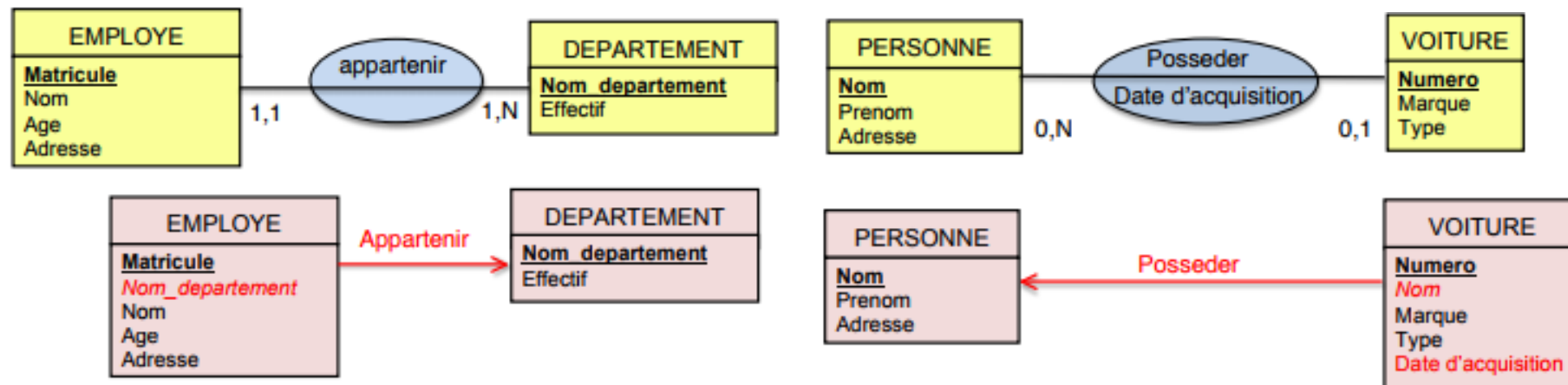
LANGAGE SQL DDL

- ❖ **Règle 1:** Toute **entité** se traduit par une **table** dans laquelle les **attributs** deviennent les **colonnes** et l'**identifiant** constitue alors la **clé primaire** de la table.



LANGAGE SQL DDL

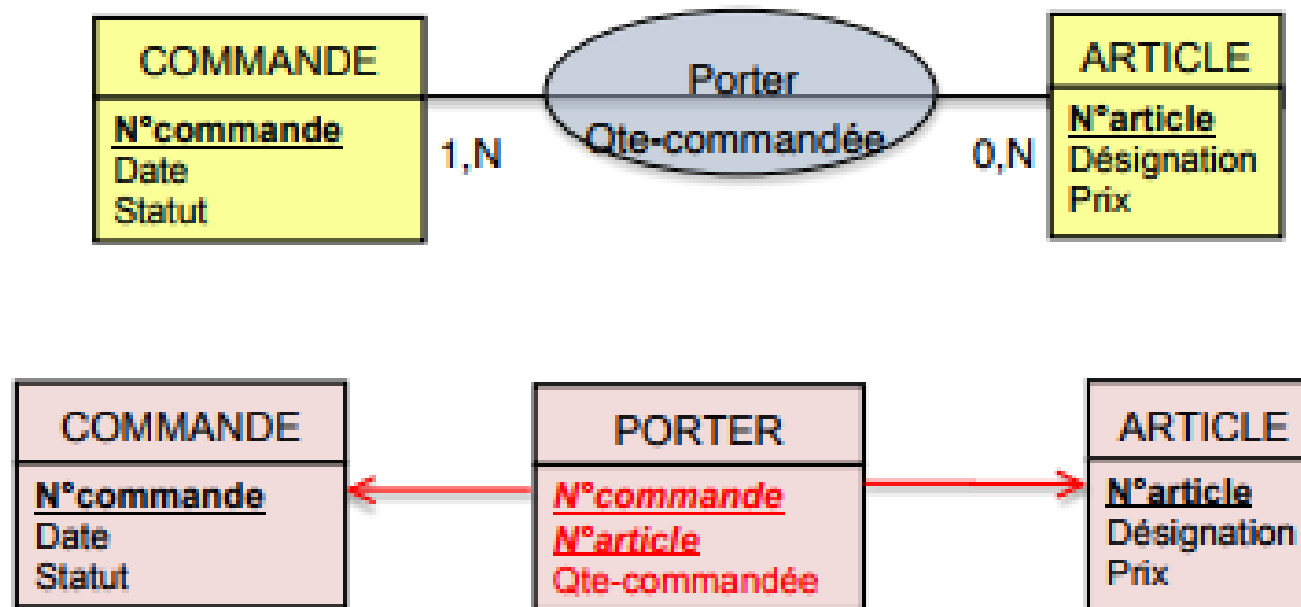
- ❖ **Règle 2:** Une association binaire **disparaît** et la clé de la relation **relative** à la **cardinalité** (0,n) ou (1,n) devient une **clé étrangère** dans la table coté **0,1** ou **1,1**.



- ❖ **Rq:** Cette clé étrangère ne peut pas recevoir la valeur null si la cardinalité est 1,1. Donc, il s'agit d'une colonne obligatoire;

LANGAGE SQL DDL

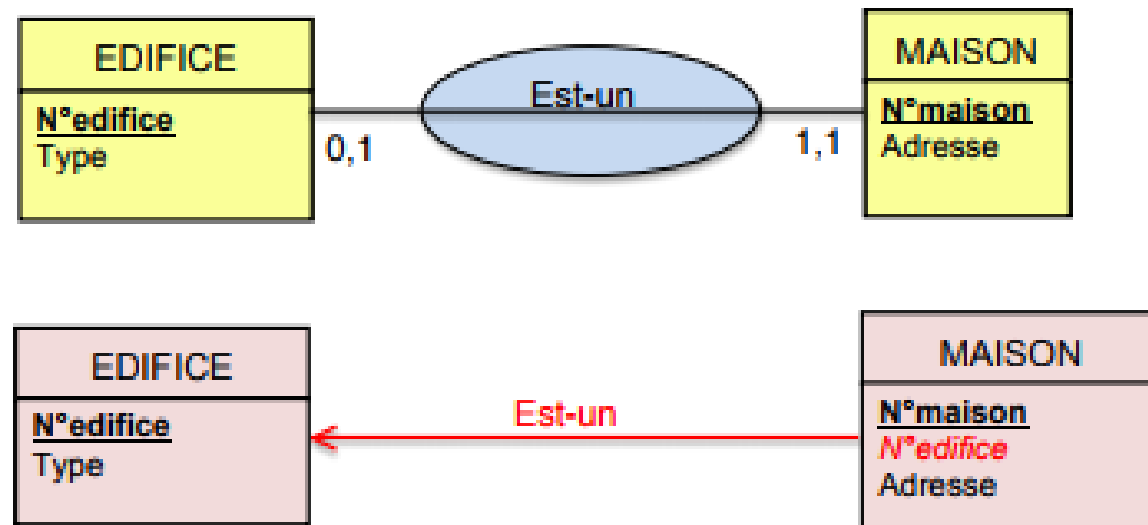
- ❖ **Règle 3:** Une association binaire de type **n : m** devient une **table supplémentaire** dont la clé primaire est **composée** des deux clés étrangères.



LANGAGE SQL DDL

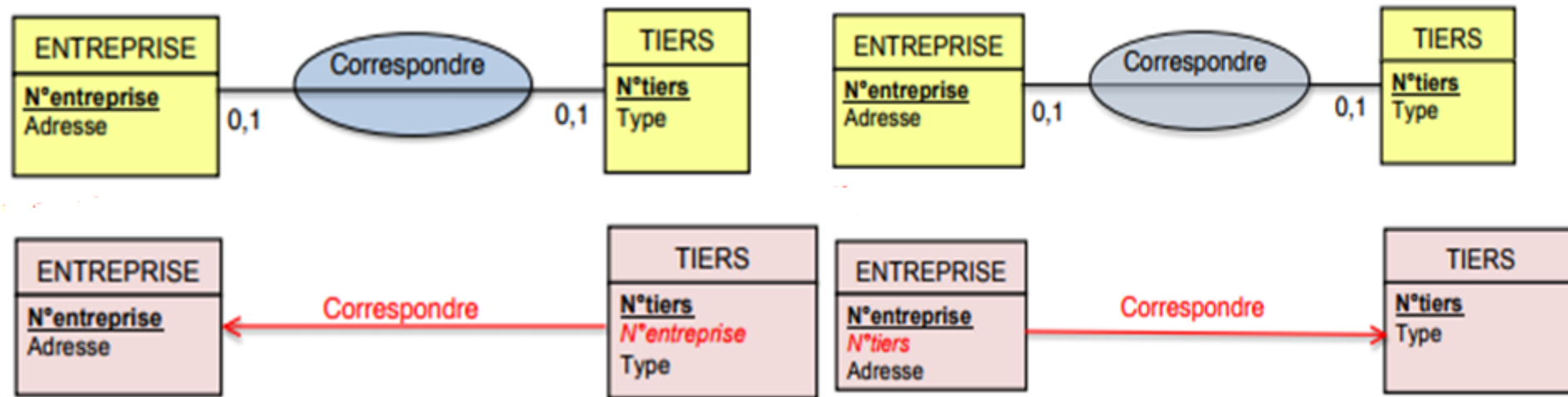
❖ **Règle 4:** L'association binaire de **type 1,1** est **la plus imposée**. Donc, on doit en plaçant une **clé étrangère** dans la table «**MAISON** ». En plus, la **clé étrangère** se voit **imposer**:

- ❖ Une contrainte d'**unicité**;
- ❖ Une contrainte d'obligation (**Not Null**).



LANGAGE SQL DDL

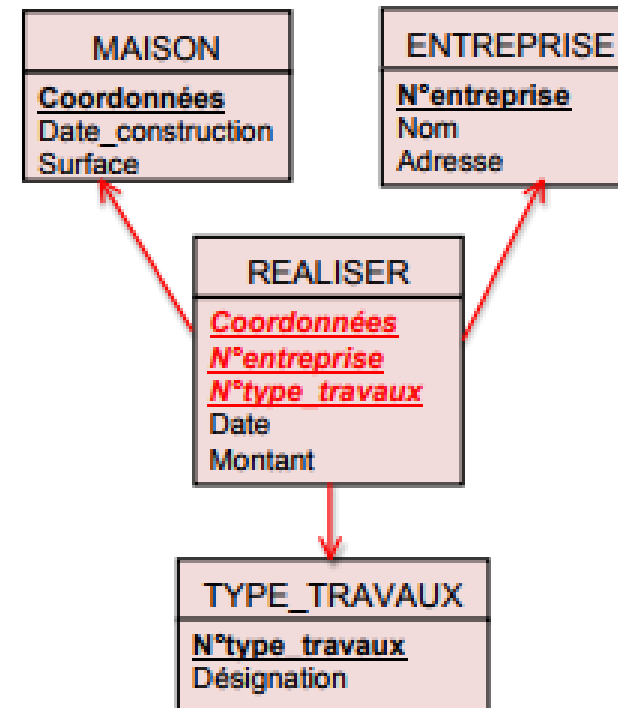
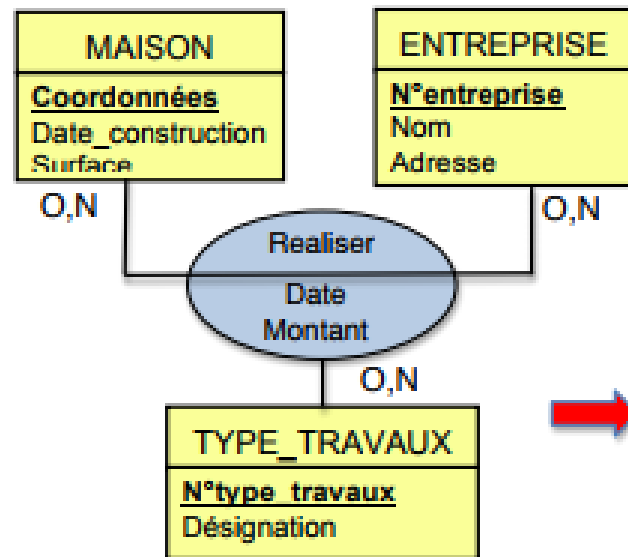
- ❖ Règle 4: il existe deux solutions pour l'association binaire de type 0:1. Mais, la plus imposée prend la cardinalité 1,1 :



- ❖ RQ: Si elles ont même niveau d'importance, on représente l'association par une table supplémentaire (même remarque pour une association binaire de type 1,1 : 1,1).

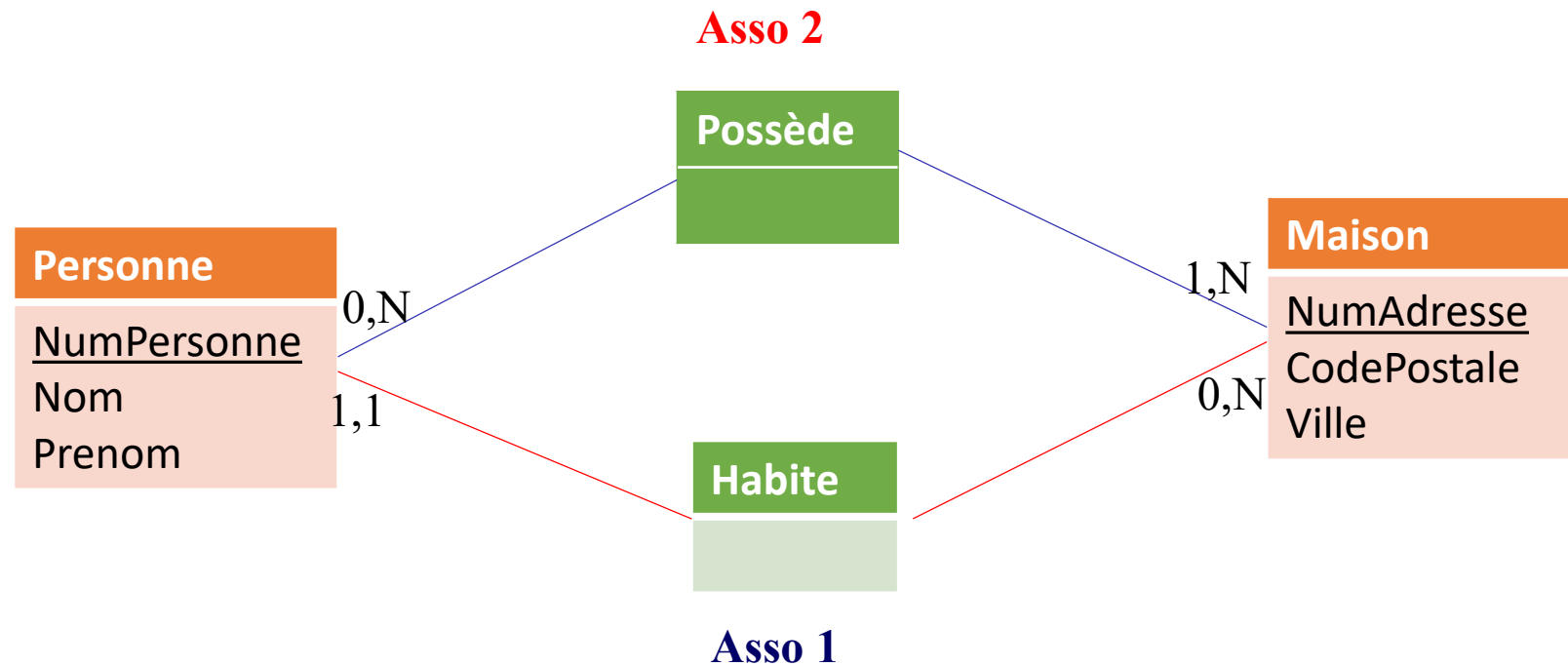
LANGAGE SQL DDL

- ❖ **Règle 5:** Une association non binaire est traduite par une **table supplémentaire** dont la **clé primaire** est **composée** d'autant de **clés étrangères** que d'entité en association.



LANGAGE SQL DDL

❖ Chercher le MLD du MCD dans les deux sens:



LANGUAGE SQL DDL

❖ Réponse: Association 1

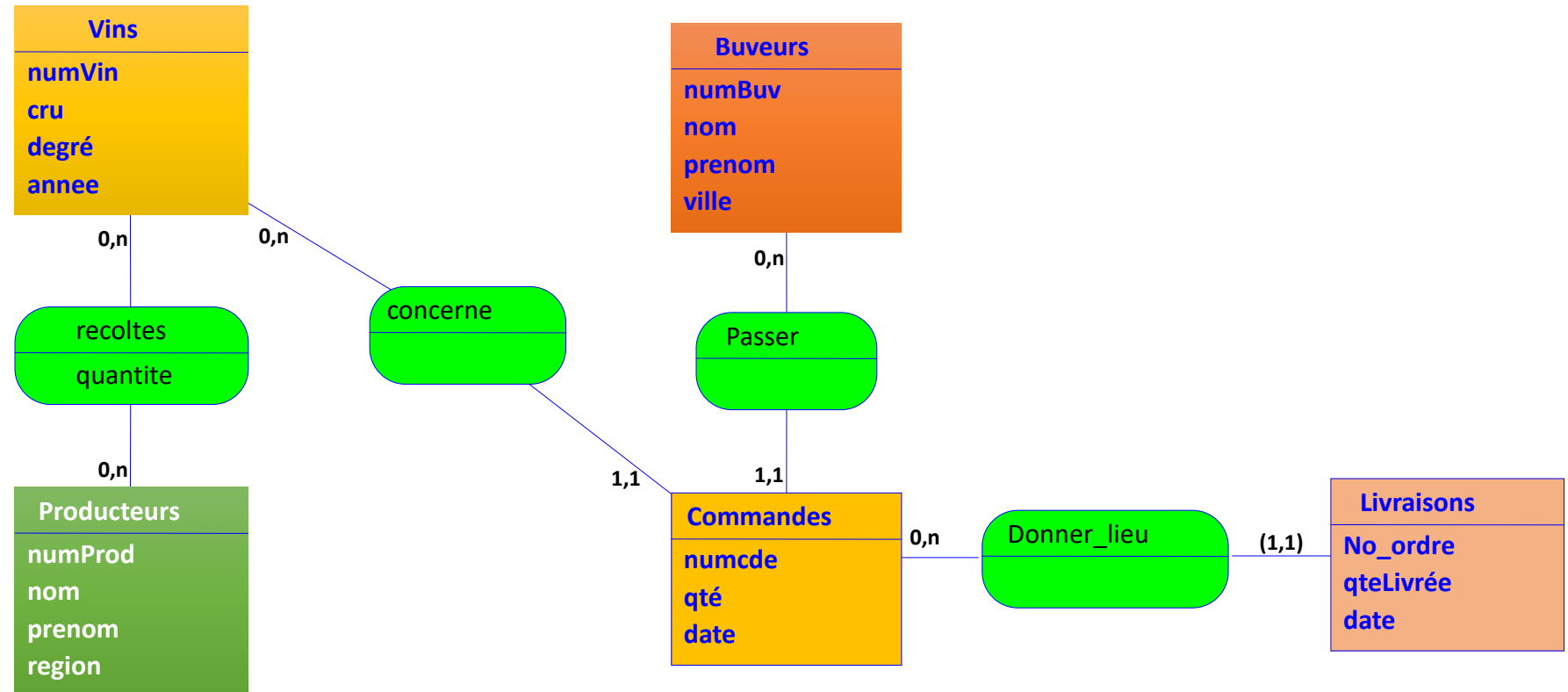
- ❖ `Personne(NumPersonne, Nom, Prenom, #NumAdresse);`
- ❖ `Maison(NumAdresse, CodePostale, Ville);`

❖ Réponse: Association 2

- ❖ `Personne(NumPersonne, Nom, Prenom);`
 - ❖ `Maison(NumAdresse, CodePostale, Ville);`
 - ❖ `Possede(#NumPersonne, #NumAdresse);`
- ❖ **Construire** les tables dans les deux cas en nommant les contraintes.

LANGAGE SQL DDL

Trouver le MLD :



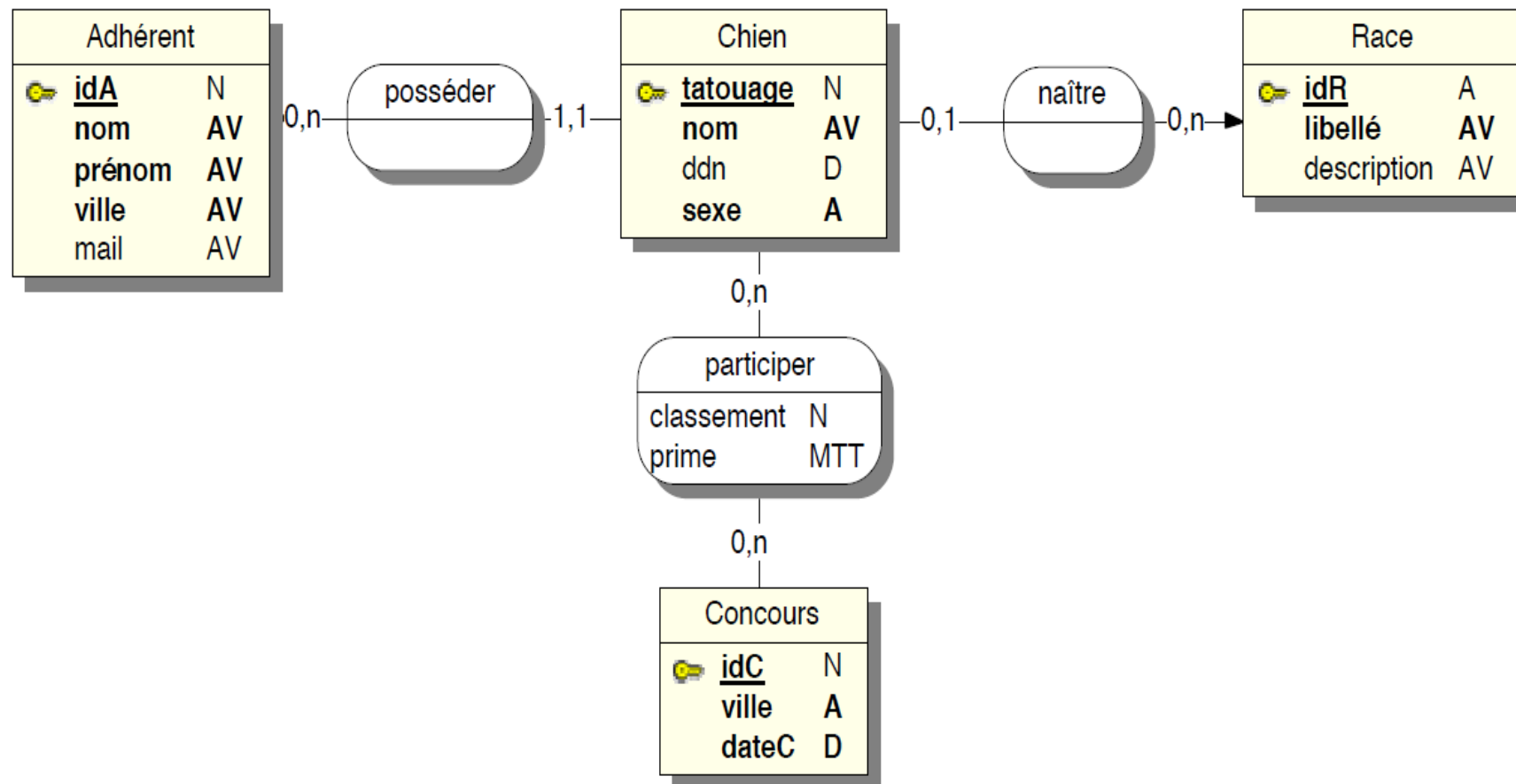
LANGAGE SQL DDL

Réponses

- Vins(NumVin, cru, annee, degre)
- Producteurs(NumProd, nom, prenom, region)
- Recoltes(#NumVin, #Numprod, quantite)
- Buveurs(NumBuy, nom, prenom, ville)
- Commandes(NumCde, date, qte, # NumBuy, # NumVin)
- Livraisons(No ordre, qteLivree, #NumCde)
- Construire les tables dans les deux cas en nommant les contraintes.

LANGAGE SQL DDL

Trouver le MLD :



LANGAGE SQL DDL

Réponses

- Adherent = (idA, nom, prenom, ville, mail);
- Race = (idR, libelle, description);
- Chien = (tatouage, nom, ddn, sexe, # idA, #idR);
- Concours = (idC, ville, dateC);
- Participation = (#tatouage, # idC, classement, prime);
- Construire les tables dans les deux cas en nommant les contraintes.

LANGAGE SQL DDL

Création des Index

Les tables sont stockées sur le disque de l'ordinateur. Si elles sont très volumineuses, **l'accès** aux données et leur **modification** risquent de prendre un temps considérable.

- ❖ Un index est une **structure physique capable d'évoluer** les performances (**temps d'exécution**) des requêtes de **lecture**, de **suppression** et de **modification**.
- ❖ Une **clé primaire**, par **défaut**, elle est **indexée** à l'aide d'un index sans **doublons** (**Unique**);
- ❖ Un index est créé par l'instruction **Create index**, modifié par la commande **Alter index... rename to ...** et supprimé par la requête **Drop Index**.

LANGAGE SQL DDL

Création des Index

Un index est une structure de données **redondante** organisée de manière à **accélérer** la **recherche** d'une **information** (**donnée**);

❖ Les SGBD crée systématiquement un index chaque fois que l'on pose :

- ❖ Une clé primaire (primary key);
- ❖ Ou bien une contrainte d'unicité (unique);

❖ L'ordre des colonnes constituant un index est important:

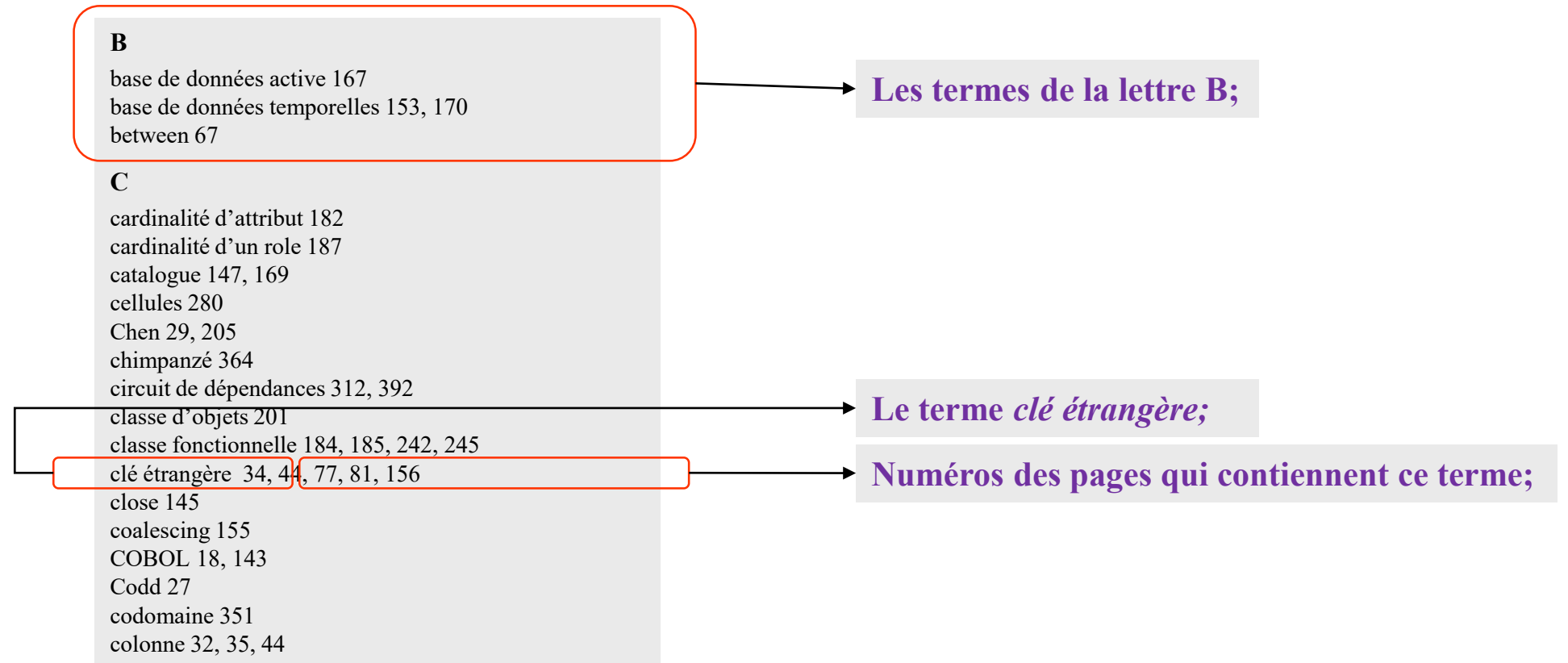
- ❖ Colonnes A, B et C $\neq$ colonnes B, C et A ;

❖ Il existe trois façons permettant de créer un index:

- ❖ Lors de la création de la table: **Create table** nom_table
- ❖ Avec la clause: **Create Index** nom_index.....
- ❖ Avec la clause: **Alter Table** nom_table **ADD Index**.....

LANGAGE SQL DDL

Index dans un livre



LANGAGE SQL DDL

Index dans une base de données

Index sur la colonne LOCALITE

Bruxelles	11
Genève	03
Lille	12
Namur	01 06 14 15
Paris	16
Poitiers	02 07 09
Toulouse	04 05 08 10 13

Numéros des lignes dont
LOCALITE = 'Toulouse'

Les valeurs de LOCALITE,
triées par ordre alphabétique

CLIENT					
	NCLI	NOM	ADRESSE	LOCALITE	(CAT) COMPTE
01	B062	GOFFIN	72, r. de la Gare	Namur	B2 -3200
02	B112	HANSENNE	23, r. Dumont	Poitiers	C1 1250
03	B332	MONTI	112, r. Neuve	Genève	B2 0
04	B512	GILLET	14, r. de l'Eté	Toulouse	B1 -8700
05	C003	AVRON	8, r. de la Cure	Toulouse	B1 -1700
06	C123	MERCIER	25, r. Lemaître	Namur	C1 -2300
07	C400	FERARD	65, r. du Tertre	Poitiers	B2 350
08	D063	MERCIER	201, bvd du Nord	Toulouse	-2250
09	F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1 0
10	F011	PONCELET	17, Clos des Erables	Toulouse	B2 0
11	F400	JACOB	78, ch. du Moulin	Bruxelles	C2 0
12	K111	VANBIST	180, r. Florimont	Lille	B1 720
13	K729	NEUMAN	40, r. Bransart	Toulouse	0
14	L422	FRANCK	60, r. de Wépion	Namur	C1 0
15	S127	VANDERKA	3, av. des Roses	Namur	C1 -4580
16	S712	GUILLAUME	14a, ch. des Roses	Paris	B1 0

Chaque ligne possède un numéro unique;
L'accès à une ligne de numéro donné est très rapide;

LANGAGE SQL DDL

Création des Index

L'utilisation d'un **index** simplifie et accélère les opérations de recherche, de triage, de jointure ou d'agrégation effectuées par le SGBD.

❖ L'**index** placé sur une table va permettre au SGBD d'accéder très rapidement aux enregistrements, selon la valeur d'un ou plusieurs champs.

❖ **Rq:** Nous pouvons créer plusieurs index sur une même table.

fldNom	fldPréno	fldAg	fldSalaire	fldService	INDEX	
Weber	Jos	34	68000 Luf	Comptabilité	●	Da Costa
Da Costa	Antonio	27	70000 Luf	Informatique	●	Feller
Feller	Emil	43	65000 Luf	Expédition	●	Weber
...	...	...	...	...		

LANGAGE SQL DDL

Création des Index

La création d'un index sur une colonne donnée se traduit par la création de **deux colonnes**:

- ❖ Une colonne contient les **références** «**adresses**» vers les enregistrements de la table;
- ❖ Une colonne contient toutes les **valeurs** de la colonne référencée de la table dans l'ordre souhaité;
- ❖ Les références définissent des **liens** vers les enregistrements de la (les) colonne indexée;

fldNom	fldPréno	fldAg	fldSalaire	fldService		INDEX
Weber	Jos	34	68000 Luf	Comptabilité	↖	Da Costa
Da Costa	Antonio	27	70000 Luf	Informatique	↗	Feller
Feller	Emil	43	65000 Luf	Expédition	↘	Weber

LANGAGE SQL DDL

Propriétés des Index

Le principe d'un index est l'association de l'adresse (**référence**) de chaque enregistrement (**RowId**) avec la **valeur** des colonnes indexées:

- ❖ Un index peut être lié à **une** ou **plusieurs** colonnes d'une table donnée;
- ❖ Un index ne peut contenir que des colonnes ayant un des types de données suivants: **Texte, Numérique ou Date/Heure**;
- ❖ Un index est **automatiquement** mis à jour par le SGBD lors:
 - ❖ D'un ajout, d'une modification ou d'une suppression d'enregistrements dans une table;
 - ❖ Ceci est transparent pour l'utilisateur de la base de données;
- ❖ Il existe deux types d'index:
 - ❖ Index **avec** doublons;
 - ❖ Index **sans** doublons;

LANGAGE SQL DDL

Propriétés des Index

- ❖ Les index ont deux inconvénients:
 - ❖ Ils prennent de la place en mémoire.
 - ❖ Ils ralentissent les requêtes d'insertion, modification et suppression, puisqu'à chaque fois, **il faut remettre l'index à jour en plus de la table.**
- ❖ Un index est créé par la commande **Create Index** en spécifiant:
 - ❖ Le nom,
 - ❖ La table à laquelle il est associé,
 - ❖ Les colonnes qui le composent,
 - ❖ L'ordre de rangement des valeurs dans l'index (par défaut **asc=ascendant**, **desc=descendant**).

LANGAGE SQL DDL

Création des Index

❖ Création d'un index:

❖ **CREATE Index Name_Index on Name_Table(NameColumn ordre,) ;**

❖ Exemple:

❖ **CREATE Index Index_NCLI on CLIENT (NCLI desc);**

❖ **Remarques:** Intérêt des index est d'accélérer les requêtes qui utilisent des colonnes indexées comme critères de recherche.

LANGAGE SQL DDL

Index sur des colonnes de Types VARCHAR

Dans cette situation, l'ordre de comparaison est par l'ordre des caractères. En effet, la chaîne des caractères est définie comme un tableau des caractères:

1re lettre	2e lettre	3e lettre	4e lettre	5e lettre	6e lettre	7e lettre	8e lettre	9e lettre	10e lettre
B	o	u	l	i	a	n			
C	a	r	a	m	o	u			
D	u	p	o	n	t				
D	u	p	o	n	t				

- **Remarque:** Il est fortement conseillé de préciser la nombre de caractères inclus dans la comparaison lors de la création d'un index : **NameColumn(Nombre de caractères);**

LANGAGE SQL DDL

Index UNIQUE ou FULLTEXT

Un index **UNIQUE** sur une colonne (ou plusieurs) permet d'ajouter une contrainte d'unicité sur les colonnes incluses dans l'index. Cela implique, qu'on aura jamais des valeurs (ou bien des combinaisons) redondante dans la table.

❖ **Exemple: Id, Login, Password: d'un site donné.**

- ❖ Il faut éviter que deux utilisateurs utilisent le même **Login**. Pour cela, on doit créer un index **UNIQUE** sur la colonne **Login** de la table **Utilisateur**.
- ❖ Un index **FULLTEXT** est utilisé pour faire des recherches de manière **puissante** et **rapide** sur un **texte**. On n'utilise donc ce type d'index que sur les colonnes de **type CHAR, VARCHAR ou TEXT**.

LANGAGE SQL DDL

Index UNIQUE

Nous pouvons définir un index simple par la clause Unique ou bien Index:

❖ Syntaxe:

```
❖ CREATE TABLE nom_table (  
    colonne1 INTEGER,  
    colonne2 VARCHAR(40) UNIQUE, -- Crée un index unique sur colonne2  
);
```

❖ La clause **UNIQUE** n'utilise pas le mot-clé **INDEX** ;

❖ Il est possible de définir des index composites (**sur plusieurs colonnes**) ;

LANGUAGE SQL DDL

Création des Index sur plusieurs colonnes

Il est possible de créer un index sur plusieurs colonnes, en séparant chaque élément par une virgule :

```
❖ CREATE TABLE nom_table (  
    colonne1 description_colonne1,  
    [colonne2 description_colonne2,  
    colonne3 description_colonne3,  
    ...,]  
    [PRIMARY KEY (colonne_clé_primaire)],  
    [INDEX [nom_index] (colonne1_index [, colonne2_index, ...])]  
)
```

LANGAGE SQL DDL

Lors de la création de la table *Client*, nous voulons indiquer un **index sur la date de naissance, et un autre sur les 10 premières lettres du nom**:

```
❖ CREATE TABLE Client(  
    Id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    Nom VARCHAR(40) NOT NULL,  
    Prenom VARCHAR(30),  
    date_naissance DATETIME NOT NULL,  
    commentaires TEXT,  
    PRIMARY KEY (Id),  
    INDEX ind_date_naissance (date_naissance),  
    INDEX ind_nom (Nom(10))).
```

Remarque: dans l'index créé sur le nom, le chiffre entre parenthèses précise le nombre de caractères pris en compte;

LANGAGE SQL DDL

Création des Index : UNIQUE ou FULLTEXT

Pour la création des index UNIQUE ou FULLTEXT, le principe reste le même:

```
❖ CREATE TABLE nom_table (  
    colonne1 INT NOT NULL,  
    colonne2 VARCHAR(40),  
    colonne3 TEXT,  
    UNIQUE [INDEX] ind_uni_col2 (colonne2),  
    FULLTEXT [INDEX] ind_full_col3 (colonne3)  
)
```

Remarque: Crée un index UNIQUE ou bien FULLTEXT sur une colonne donnée, la clause INDEX est **facultative**.

LANGUAGE SQL DDL

Création des Index

❖ Exemple:

```
❖ CREATE TABLE Client(  
    Id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    Nom VARCHAR(40) NOT NULL,  
    Prenom VARCHAR(30),  
    date_naissance DATETIME NOT NULL,  
    commentaires TEXT,  
    PRIMARY KEY (Id),  
    INDEX ind_date_naissance (date_naissance),  
    INDEX ind_nom (nom(10)),  
    UNIQUE INDEX ind_uni_nom_prenom(nom, prenom),  
    FULLTEXT INDEX ind_uni_nom_prenom(nom, commentaires )  
)
```

LANGAGE SQL DDL

Création des Index sur une table

Il existe deux commandes permettant de créer des index sur une table existante. Ces deux commandes sont équivalentes, la seule différence est au niveau de la syntaxe :

- ❖ ALTER TABLE

- ❖ CREATE INDEX.

- ❖ Ajout avec ALTER TABLE:

- ❖ ALTER TABLE nom_table

- ADD INDEX [nom_index] (colonne_index [, colonne2_index ...]);

- Ajout d'un index simple

- ❖ ALTER TABLE nom_table

- ADD UNIQUE [nom_index] (colonne_index [, colonne2_index ...]);

- Ajout d'un index UNIQUE

- ❖ ALTER TABLE nom_table

- ADD FULLTEXT [nom_index] (colonne_index [, colonne2_index ...]);

- Ajout d'un index FULLTEXT

LANGAGE SQL DDL

Création des Index sur une table

❖ Ajout d'un Index avec **CREATE INDEX**:

❖ La syntaxe :

❖ **CREATE INDEX** nom_index
ON nom_table (colonne_index [, colonne2_index ...]);

-- Crée un index simple

❖ **CREATE UNIQUE INDEX** nom_index
ON nom_table (colonne_index [, colonne2_index ...]);

-- Crée un index UNIQUE

❖ **CREATE FULLTEXT INDEX** nom_index
ON nom_table (colonne_index [, colonne2_index ...]);

-- Crée un index FULLTEXT

LANGUAGE SQL DDL

❖ Exemples:

❖ Pour ALTER TABLE:

❖ ALTER TABLE Client
ADD INDEX ind_nom (nom);

❖ Pour Create Index:

❖ Create index Ind_Nom
on Client (nom);

LANGAGE SQL DDL

Création des Index Unique sur une table

Un index **UNIQUE** crée implicitement une **contrainte** s'elle n'est pas été **nommée**. Ainsi, vous pouvez explicitement créer cette contrainte:

❖ Lors de la création d'une table:

```
CREATE TABLE nom_table (  
    colonne1 INT NOT NULL,  
    colonne2 VARCHAR(40),  
    colonne3 TEXT,  
    CONSTRAINT [Nom_contrainte] UNIQUE [INDEX] ind_uni_col2 (colonne2) );
```

❖ Par la commande Alter table:

```
ALTER TABLE nom_table  
    ADD CONSTRAINT [Nom_contrainte] UNIQUE ind_uni_col2 (colonne2);
```

LANGUAGE SQL DDL

❖ Suppression d'un index:

❖ **ALTER TABLE** nom_table
DROP INDEX nom_index;

DUT-Informatique –S3

2021/2022



Langage LMD

Langage MDL comporte deux grandes familles de fonctions permettant l'extraction et la modification de données.

- ❖ **Extraction**: La **requête select** définit la clause la plus riche. Elle permet l'extraction de données qui répondent à des questions posées par les utilisateurs.
- ❖ **Modification du contenu**: Elle comporte trois commandes permettant **l'insertion (insert)**, la **suppression (delete)** et la **modification (update)** des lignes d'une table.

Dans la suite, nous aborderons les pointes suivants :

- ❖ Insertion des données;
- ❖ Extraction de données d'une seule table;
- ❖ Extraction de données regroupées;
- ❖ Modification des données ;
- ❖ Suppression de données.
- ❖ Extraction de données d'une seule table, mais en corrélation avec les données des autres tables;
- ❖ Extraction de données de plusieurs tables;

Langage LMD

Commande Insert Into

La commande **insert into** permet d'ajouter des enregistrements à une table d'une base de données. On distingue trois types d'insertions:

- ❖ Des valeurs nouvelles;
- ❖ Des valeurs calculées à partir d'une expression arithmétique;
- ❖ Des valeurs extraites par une requête **SFW**;

Pour ajouter une nouvelle ligne à une table donnée, on utilise l'instruction **insert into ... values;**

❖ **Syntaxe:**

```
INSERT INTO nom_table [(nom_col1, nom_col2,...)] VALUES (val1, val2...);
```

❖ **Exemple:**

```
INSERT INTO Etudiant (Num_Et, Nom, Prenom)  
VALUES(96035, 'AYADI', 'Morad');
```

Langage LMD

Commande Insert Into

Créer une table nommée **Client** qui contient les attributs suivants :

(NCLI, NOM, LOCALITE, CAT, COMPTE)

❖ Puis insérer les lignes suivantes dans la table **Client**:

CLIENT					
NCLI	NOM	ADRESSE	LOCALITE	(CAT)	COMPTE
B062	GOFFIN	72, r. de la Gare	Namur	B2	-3200
B112	HANSENNE	23, r. Dumont	Poitiers	C1	1250
B332	MONTI	112, r. Neuve	Genève	B2	0
B512	GILLET	14, r. de l'Eté	Toulouse	B1	-8700
C003	AVRON	8, r. de la Cure	Toulouse	B1	-1700
C123	MERCIER	25, r. Lemaître	Namur	C1	-2300
C400	FERARD	65, r. du Tertre	Poitiers	B2	350
D063	MERCIER	201, bvd du Nord	Toulouse		-2250
F010	TOUSSAINT	5, r. Godefroid	Poitiers	C1	0
F011	PONCELET	17, Clos des Erables	Toulouse	B2	0
F400	JACOB	78, ch. du Moulin	Bruxelles	C2	0
K111	VANBIST	180, r. Florimont	Lille	B1	720
K729	NEUMAN	40, r. Bransart	Toulouse		0
L422	FRANCK	60, r. de Wépion	Namur	C1	0
S127	VANDERKA	3, av. des Roses	Namur	C1	-4580
S712	GUILLAUME	14a, ch. des Roses	Paris	B1	0

❖ **Rq:** Si toutes les colonnes de la table sont concernées **liste_colonnes** peut-être négligée.

Langage LMD

Commande Insert Into

L'ordre et le type des valeurs sont **très importants**. Vous devez respecter l'ordre et le type de création des colonnes lors de la création de la table;

❖ **Exemple:**

```
Insert Into CLIENT (NCLI,NOM, COMPTE,LOCALITE) Values ('C402','ASIMI ',  
2500,'Guelmim')
```

Il est possible d'insérer dans une table existante des **données extraites d'une ou plusieurs tables**. Pour cet intérêt, vous devez combiner les deux clauses: **insert et select**;

❖ **Exemple:**

❖ Nous voulons ajouter à la table CLIENT_GUELMIM (NUMERO, NOM, COMPTE) les données extraites de **la table** CLIENT :

```
Insert Into CLIENT_GUELMIM Select NCLI, NOM, COMPTE From CLIENT Where  
LOCALITE = 'GUELMIM';
```

Langage LMD

Commande Insert Into

Si vous avez oublié de spécifier la valeur d'une colonne donnée, elle va prendre soit:

- ❖ **la valeur null, ou la valeur par défaut** (s'elle est définie avec la clause default);
- ❖ si la colonne est obligatoire (**not null**):
 - ❖ Elle doit recevoir une valeur;
 - ❖ Une valeur par défaut, s'elle est déclarée par défaut (**default**);
- ❖ **Rq:** lors d'insertion des données dans une table donnée, vous devez respecter les contraintes d'intégrité:
 - ❖ **Identifiants,**
 - ❖ **Intégrité référentielle,**
 - ❖ **Colonnes obligatoires;**

Langage LMD

Commande Insert Into

Lors d'insertion, vous devez respecter **la liste de colonnes en nombre et en type**. Cette Liste de Valeurs peut contenir:

- ❖ des valeurs **textuelles**;
- ❖ des expressions **numériques**;
- ❖ des valeurs **nulles** ;
- ❖ des valeurs par **défaut**.

❖ Remarques:

- ❖ L'opérateur ***** permet d'afficher tous les champs définis dans la table;
- ❖ Les valeurs sont séparées par **des virgules**;
- ❖ Les données du type **TEXTE** sont entourées **d'apostrophes**.

Langage LMD

Commande Select

La requête **Select** est la plus riche. Elle prend plusieurs formes. Elle définit deux types de clauses:

- ❖ **Obligatoires (SF)** et Facultatives (WGH).
- ❖ Nous donnons premièrement la forme générale de cette requête, puis, nous allons détailler l'utilité de chaque clause:
 - ❖ **SELECT** [ALL | DISTINCT] { * | expression [AS nom_affiché] } [, ...]
 - ❖ **FROM** nom_table [[AS] alias] [, ...]
 - ❖ [**WHERE** prédicat]
 - ❖ [**GROUP BY** expression [, ...]]
 - ❖ [**HAVING** condition [, ...]]
 - ❖ [{ **UNION** | **INTERSECT** | **EXCEPT** [**ALL**] } requête]
 - ❖ [**ORDER BY** expression [ASC | DESC] [, ...]]

Langage LMD

Commande Select

La requête **Select** permet l'extraction simple de toutes les lignes d'une table donnée. Elle affiche une table entièrement:

```
SELECT * FROM Nom_table;
```

Exemple:

```
SELECT * FROM Client;
```

- ❖ Ici, le symbole ***** signifie toutes les **lignes**. Plutôt, dans quelques situations, cette requête peut engendrer des lignes dupliquées:

```
SELECT NOM, LOCALITE FROM Client Where CAT= ' B1 ';
```

- ❖ La solution est d'utiliser la clause **distinct** qui élimine les lignes doubles retournées par une requête Select:

```
Select distinct NOM, LOCALITE from CLIENT where CAT = ' B1 ';
```

Langage LMD

Valeur Null

Une valeur **NULL** est une valeur non disponible, non affectée, inconnue ou inapplicable;

- ❖ Une valeur **NULL** est différente du **zéro** ou de **l'espace**;
- ❖ Le **zéro** est un **chiffre** et **l'espace** est un **caractère**;

Exemple :

- ❖ **SELECT * FROM Client WHERE CAT IS NULL;**
- ❖ **SELECT * FROM Client WHERE COMPTE >0;**

Langage LMD

Alias

Un alias est un nom qui suit immédiatement le nom de la colonne avec le mot-clé **AS**. Il sert à:

- ❖ Renommer l'en-tête d'une colonne ou des expressions arithmétiques;

Il doit obligatoirement être inclus entre **guillemets** :

- ❖ s'il contient des **espaces** ou des caractères spéciaux;
- ❖ Egalement, si on veut tenir compte des **majuscules/minuscules**.

❖ Exemple :

❖ **SELECT** nom, (2*note1+ 3*note2)/5 **AS** " **Notes Info** " **FROM** Etudiant;

Il est possible d'utiliser des alias des tables utilisé dans une jointure pour simplifier l'écriture de la requête select:

❖ Exemple :

❖ **SELECT** nomE, nomF **FROM** Etudiant e, Filiere f
WHERE e.codeE= f.codeE;

Langage LMD

Concaténation

Elle s'écrit avec deux **barres verticales** (**||**). Elle permet de concaténer des expressions (**colonnes, calculs, fonctions ou constantes**). La colonne résultante est considérée comme une chaîne de caractères:

❖ **Exemple :**

❖ **SELECT** nom **||** ' a la moyenne ' **||(2*note1+ 3*note2)/5 AS " note Info " FROM Etudiant;**

Application:

- ❖ **SQL> create** table **Etudiant**(nom varchar2(30) **primary key**, note1 integer, note2 integer);
- ❖ **SQL> insert into Etudiant values**('Younes', 13, 14);
- ❖ **SQL> insert into Etudiant values**('Nabil', 15, 14);
- ❖ **SQL> insert into Etudiant values**('Jalal', 15, 15);

Langage LMD

Concaténation

```
SQL> select nom||' a la moyenne '||(2*note1+3*note2)/5 As " Note Info" From  
Etudiant;
```

❖ Note Info

- ❖ Younes a la moyenne 13,6
- ❖ Nabil a la moyenne 14,4
- ❖ Jalal a la moyenne 15

Langage LMD

Pseudo-colonne ROWNUM

La pseudo-colonne **ROWNUM** retourne un entier indiquant l'ordre séquentiel de chaque enregistrement extrait par la requête.

```
SQL> Select rownum, nom||' a la moyenne '||(2*note1+3*note2)/5 As "note Info"  
      From Etudiant;
```

❖ **ROWNUM** **Note Info**

- 1 Younes a la moyenne 13,6
- 2 nabil a la moyenne 14,4
- 3 Jalal a la moyenne 15

Langage LMD

Séquences

Une **séquence** est un **objet virtuel** qui ne contient aucune donnée mais qui s'utilise pour générer **automatiquement des valeurs (NUMBER)**.

- ❖ Elles sont utiles pour composer des clés primaires de tables quand vous ne disposez pas de colonnes adéquates à cet effet.
- ❖ La commande **CREATE SEQUENCE** permet de créer une séquence dans votre schéma.
 - ❖ **CREATE SEQUENCE** *nomSéquence*

[**INCREMENT BY** *entier* / [**START WITH** *entier*]

[{ **MAXVALUE** *entier* | **NOMAXVALUE** }]

[{ **MINVALUE** *entier* | **NOMINVALUE** }]

[{ **CYCLE** | **NOCYCLE** }][{ **CACHE** *entier* | **NOCACHE** }]

[{ **ORDER** | **NOORDER** }];

Langage LMD

Séquences

- ❖ **INCREMENT BY** : donne l'intervalle entre deux valeurs de la séquence (entier positif ou négatif mais pas nul). L'intervalle par défaut est 1.
- ❖ **START WITH** : précise la première valeur de la séquence à générer.
- ❖ **MAXVALUE** : donne la valeur maximale de la séquence (ne pas dépasser $10^{29}-1$).
 - ❖ Cette limite doit être supérieure ou égale à l'entier défini dans START WITH et supérieure à MINVALUE.
- ❖ **NOMAXVALUE** : (par défaut) fixe le maximum à $10^{29}-1$ pour une séquence ascendante et à -1 pour une séquence descendante.
- ❖ **MINVALUE** : précise la valeur minimale de la séquence (ne pas dépasser la valeur $-10^{27}-1$).
- ❖ **NOMINVALUE** : (par défaut) fixe le minimum à -1 pour une séquence ascendante et à la valeur $-10^{27}-1$ pour une séquence descendante;



Langage LMD

Séquences

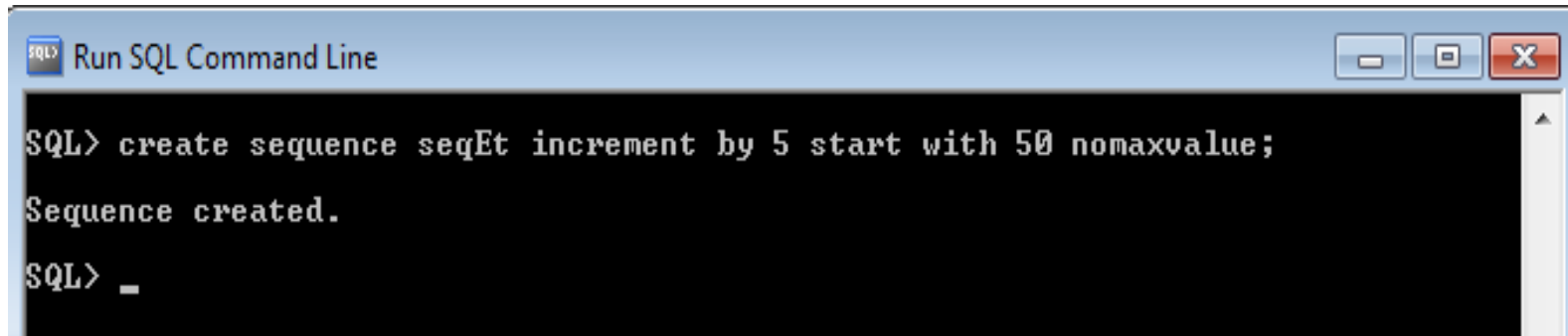
- ❖ **CYCLE** : indique que la séquence doit continuer de générer des valeurs même après avoir atteint sa limite (NOCYCLE (par défaut)).
- ❖ **CACHE** : spécifie le nombre de valeurs de la séquence que le cache doit contenir (minimum est 2).
- ❖ **NOCACHE** : indique qu'aucune valeur de la séquence n'est dans le cache.
- ❖ **ORDER** : garantit que les valeurs de la séquence sont générées dans l'ordre des requêtes.
- ❖ **NOORDER** : (défaut) ne prend pas en compte l'option précédente.

Langage LMD

Séquences

Une fois créée, une séquence *seqEt* ne peut se manipuler que via deux directives :

- ❖ *seqEt.CURRVAL* qui retourne la valeur courante de la séquence;
- ❖ *seqEt.NEXTVAL* qui incrémente la valeur de la séquence et retourne la nouvelle valeur.
- ❖ **Création d'une séquence:**

A screenshot of a 'Run SQL Command Line' window. The window has a title bar with a small SQL icon and standard minimize, maximize, and close buttons. The main area is a black terminal with white text. It shows the command 'SQL> create sequence seqEt increment by 5 start with 50 nomaxvalue;' followed by the response 'Sequence created.' and a new prompt 'SQL> _'.

```
SQL> create sequence seqEt increment by 5 start with 50 nomaxvalue;  
Sequence created.  
SQL> _
```

Langage LMD

Séquences

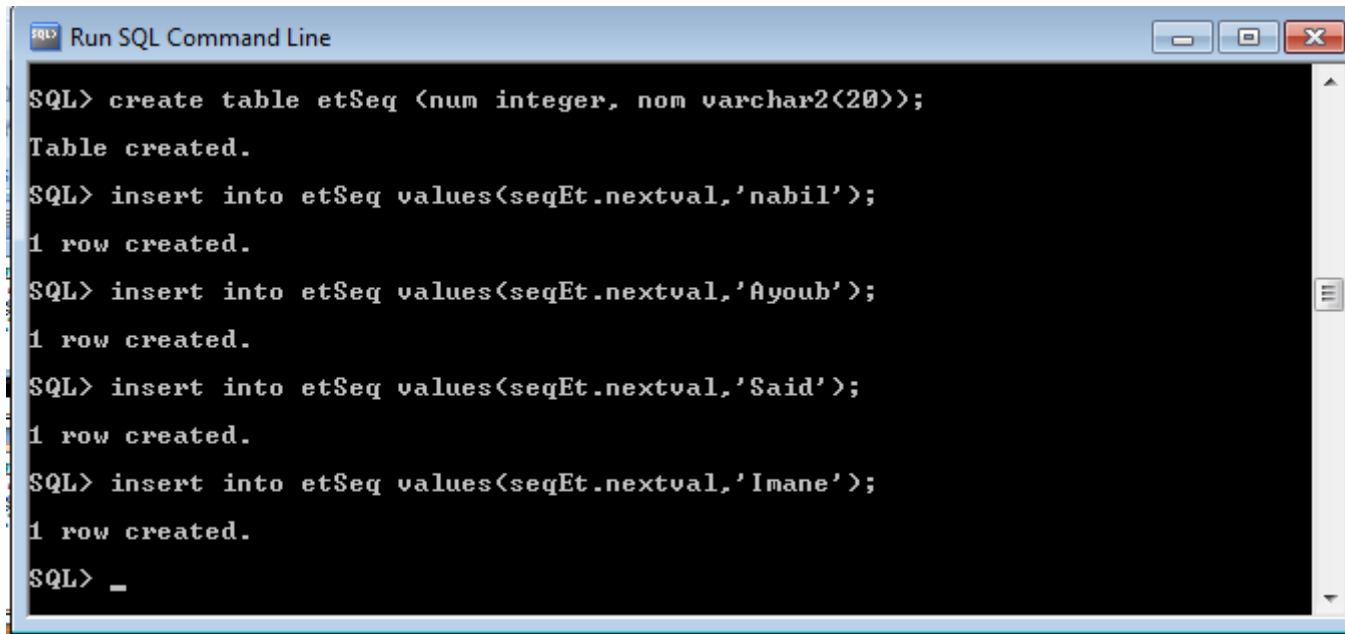
Une séquence peut être utilisée dans:

- ❖ Une Requête **Select**;
- ❖ La clause **SELECT** d'une instruction **INSERT** (Insertion multilignes);
- ❖ La clause **VALUES** d'une instruction **INSERT**;
- ❖ La clause **SET** d'une instruction **UPDATE**.

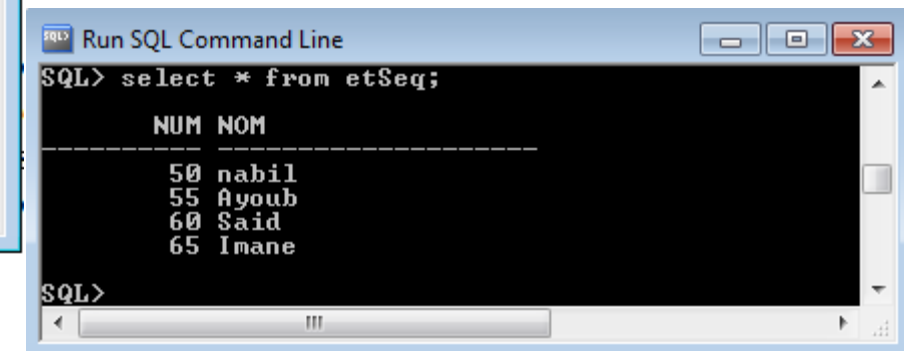
Introduction

Séquences

❖ Utilisation d'une séquence dans une requête **Insert**:



```
SQL> create table etSeq (num integer, nom varchar2(20));
Table created.
SQL> insert into etSeq values(seqEt.nextval,'nabil');
1 row created.
SQL> insert into etSeq values(seqEt.nextval,'Ayoub');
1 row created.
SQL> insert into etSeq values(seqEt.nextval,'Said');
1 row created.
SQL> insert into etSeq values(seqEt.nextval,'Imane');
1 row created.
SQL> _
```



```
SQL> select * from etSeq;
```

NUM	NOM
50	nabil
55	Ayoub
60	Said
65	Imane

```
SQL>
```

Langage LMD

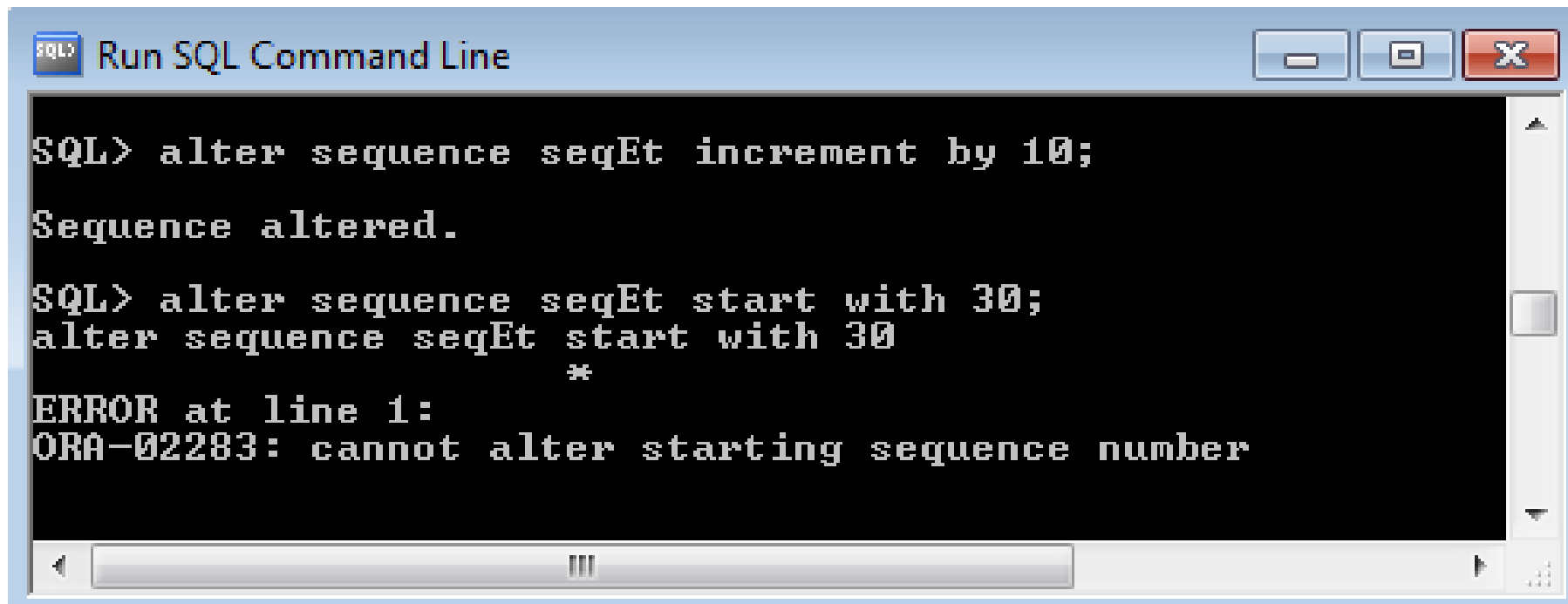
Modification d'une Séquence

La syntaxe de modification d'une séquence reprend la plupart des éléments de sa création.

- ❖ **ALTER SEQUENCE** *nomSéquence*
 - ❖ **[INCREMENT BY** *entier* **]**
 - ❖ **[{ MAXVALUE** *entier* **| NOMAXVALUE }]**
 - ❖ **[{ MINVALUE** *entier* **| NOMINVALUE }]**
 - ❖ **[{ CYCLE | NOCYCLE }]**
 - ❖ **[{ CACHE** *entier* **| NOCACHE }]**
 - ❖ **[{ ORDER | NOORDER }] ;**
- ❖ La clause **START WITH** ne peut être modifiée sans supprimer et recréer la séquence.
- ❖ La valeur **MAXVALUE** ne peut pas être affectée à une valeur plus petite que la valeur courante de la séquence.

Langage LMD

Modification d'une Séquence



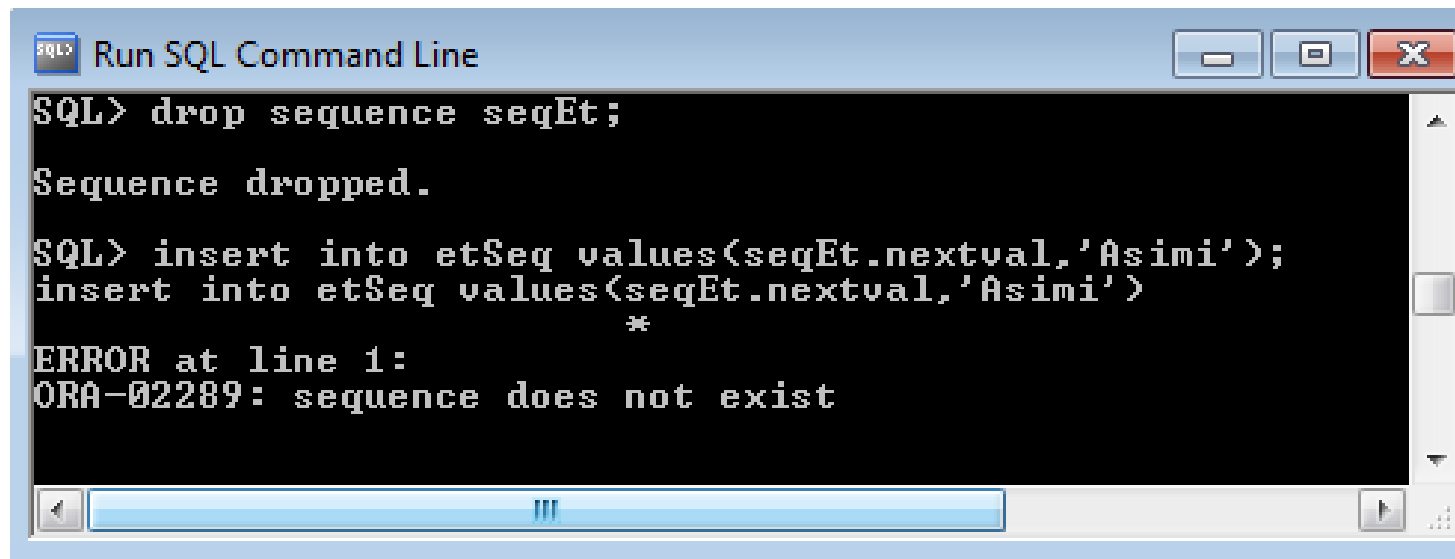
```
SQL> alter sequence seqEt increment by 10;
Sequence altered.

SQL> alter sequence seqEt start with 30;
alter sequence seqEt start with 30
      *
ERROR at line 1:
ORA-02283: cannot alter starting sequence number
```

Langage LMD

Suppression d'une Séquence

- ❖ La syntaxe de suppression d'une séquence est la suivante.
 - ❖ **DROP SEQUENCE** *nomSéquence* ;
- ❖ On supprime la séquence de notre schéma par l'instruction suivante :
 - ❖ **DROP SEQUENCE** seqEt;



```
SQL> drop sequence seqEt;

Sequence dropped.

SQL> insert into etSeq values(seqEt.nextval,'Asimi');
insert into etSeq values(seqEt.nextval,'Asimi')
                        *
ERROR at line 1:
ORA-02289: sequence does not exist
```

Langage LMD

Application - Séquence

- ❖ Créer une séquence nommée SeqEtud qui :
 1. Incrémente les valeurs par 5 ;
 2. Commence par la valeur 100 ;
 3. N'a pas d'une valeur maximale



Run SQL Command Line

```
SQL> create sequence SeqEt increment by 5 start with 100 nomaxvalue;  
  
Sequence created.
```

Langage LMD

Application - Séquence

- ❖ Créer la table EtdSeq(NEtud entier, Nom chaine, prenom chaine) :
 1. Insérer trois lignes dans cette table en utilisant la séquence;
 2. Changer l'intervalle entre deux valeurs de la séquence à 10 ;
 3. Supprimer la séquence créée ;

 Run SQL Command Line

```
SQL> create table EtdSeq (netud integer, Nom varchar2(50), prenom varchar2(40));
Table created.

SQL> insert into EtdSeq values (SeqEt.nextval, 'asimi', 'younes');
1 row created.

SQL> insert into EtdSeq values (SeqEt.nextval, 'Ayadi', 'kamal');
1 row created.

SQL> insert into EtdSeq values (SeqEt.nextval, 'Elabalmani', 'chaki');
1 row created.
```

Langage LMD

Application - Séquence

SQL> Run SQL Command Line

```
SQL> alter sequence SeqEt increment by 10;
Sequence altered.

SQL> alter sequence SeqEt increment by 5;
Sequence altered.

SQL> alter sequence SeqEt minvalue 50;
Sequence altered.

SQL> alter sequence SeqEt maxvalue 50;
alter sequence SeqEt maxvalue 50
*
ERROR at line 1:
ORA-04004: MINVALUE must be less than MAXVALUE
```

SQL> Run SQL Command Line

```
SQL> select 'la valeur courante de la sequence est' || seqet.currval from etdseq;

'LAVALEURCOURANTEDELASEQUENCEEST' || SEQET.CURRVAL
-----
la valeur courante de la sequence est110
la valeur courante de la sequence est110
la valeur courante de la sequence est110

SQL> select 'la valeur suivante de la sequence est' || seqet.nextval from etdseq;

'LAVALEURSUIVANTEDELASEQUENCEEST' || SEQET.NEXTVAL
-----
la valeur suivante de la sequence est115
la valeur suivante de la sequence est120
la valeur suivante de la sequence est125
```

Langage LMD

Algèbre relationnelle

Il s'agit d'une collection d'opérations formelles qui agissent sur des relations et produisent une relation en résultat. Ces opérations (sept) permettent d'exprimer toutes les requêtes sous forme d'expressions algébriques. On distingue deux classes d'opérateurs :

- ❖ **Opérateurs ensemblistes:** ces opérations binaires construisent à partir de deux relations une troisième relation:
 - ❖ l'union ($\cup$), l'intersection ($\cap$), la différence ($-$) et le produit cartésien ($\times$).
- ❖ **Opérateurs relationnels:**
 - ❖ les opérations **unaires de projection** (Π) et **restriction ou bien sélection** (σ) qui, à partir d'une relation, en construisant une autre,
 - ❖ et **l'opération binaire de jointure** ($\bowtie$).

Langage LMD

Union

L'opération d'union consiste en la mise en commun (**sans doublons**) des enregistrements de chaque relation ayant mêmes champs. **Les enregistrements identiques ne sont intégrés qu'une seule fois.** L'union est représentée par le caractère « U ».

- Soient R1 et R2 deux relations de même schéma:

R1	A	B	C
	a	c	10
	c	d	15
	x	y	20

R2	A	B	C
	f	g	10
	c	d	15
	m	p	12

- Union** $T = (R1 \cup R2)$ ou bien $T = \text{Union}(R1, R2)$

T	A	B	C
	a	c	10
	c	d	15
	x	y	20
	f	g	10
	m	p	12

Exemple : Le **NSS** de toutes les personnes:

```
SELECT NSS FROM étudiant
```

```
UNION
```

```
SELECT NSS FROM employe;
```

Langage LMD

Union

E1 : Enseignants

n° enseignant	nom_enseignant
1	DUPONT
3	DURAND
4	MARTIN
5	BERTRAND

E2 : Enseignants

n°enseignant	nom_enseignant
1	DUPONT
4	MARTIN
6	MICHEL

Remarques:

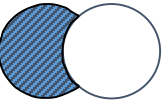
- Les deux relations doivent avoir le même nombre d'attributs définis dans le même domaine;
- La relation résultat possède les attributs des relations d'origine et les lignes de chacune, avec élimination des **doublons**.

R1 = UNION (E1, E2)

n°enseignant	nom_enseignant
1	DUPONT
3	DURAND
4	MARTIN
5	BERTRAND
6	MICHEL

Langage LMD

Différence



L'opération différence consiste à désigner les enregistrements qui appartiennent à une relation sans appartenir à l'autre ayant mêmes champs.

Syntaxe :

NomR1_NomR2_Diff(les lignes de R1≠R2) ;

- Soient R1 et R2 deux relations de même schéma:

R1	A	B	C
	a	c	10
	c	d	15
	x	y	20

R2	A	B	C
	f	g	10
	c	d	15
	m	p	12

- **Différence** $T = (R1 - R2)$ ou bien $T = \text{Minus}(R1, R2)$; 

T	A	B	C
	a	c	10
	x	y	20

```
SELECT * FROM relation_1 EXCEPT SELECT * FROM relation_2.....;
```

Langage LMD

Différence

- Soient E1 et E2 deux relations de même schéma:

E1

n° enseignant	nom_enseignant
1	DUPONT
3	DURAND
4	MARTIN
5	BERTRAND

E2

n°enseignant	nom_enseignant
1	DUPONT
4	MARTIN
6	MICHEL

- Différence $T = (E1 - E2)$ ou bien $T = \text{Minus}(E1, E2)$;

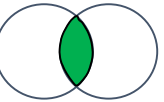
n°enseignant	nom_enseignant
3	DURAND
5	BERTRAND

Remarque:

- La relation résultat possède les attributs des relations d'origine et les lignes de la **première relation** qui n'appartiennent pas à la **deuxième**.

Langage LMD

Intersection



L'opération intersection donne les enregistrements qui sont communs entre deux relations ayant mêmes champs.
L'intersection est représentée par le caractère « $\cap$ ».

Syntaxe :

NomR1_NomR2_Inter(Champs de R1=R2) ;

- Soient R1 et R2 deux relations de même schéma:

R1	A	B	C
	a	c	10
	c	d	15
	x	y	20

R2	A	B	C
	f	g	10
	c	d	15
	m	p	12

Intersection:

$T = (R1 \cap R2)$ ou bien $T = \text{inter}(R1, R2);$
AND (R, S);

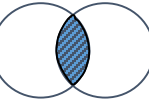
↔

T	A	B	C
	c	d	15

SELECT * FROM relation_1 INTERSECT SELECT * FROM relation_2.....;

Langage LMD

Intersection



- Soient E1 et E2 deux relations de même schéma:

E1

n° enseignant	nom_enseignant
1	DUPONT
3	DURAND
4	MARTIN
5	BERTRAND

E2

n°enseignant	nom_enseignant
1	DUPONT
4	MARTIN
6	MICHEL

Intersection: $T = (R1 \cap R2)$ ou bien $T = \text{inter}(R1, R2)$;

n°enseignant	nom_enseignant
1	DUPONT
4	MARTIN

Remarque:

- La relation résultat possède les attributs des relations d'origine et les lignes communs à chacune.

Langage LMD

Projection

La projection d'une relation consiste en la mise en place d'une **nouvelle relation en ne retenant que certaines colonnes (attributs) et en supprimant les occurrences en double (select distinct...)**.

La projection consiste à extraire certains champs de la relation, ce qui donne à cette dernière **un degré inférieur** à la **relation** de **départ**.

Syntaxe : Nom_Relation_proj(les champs à sélectionnés) ;

Soit R1 une relation,

R1	A	B	C
	a	c	10
	c	d	15
	x	y	20
	c	d	35
	x	y	60

Projection : de R1 sur les attributs A et B

$$T = P_{A, B,}(R1)$$



T	A	B
	a	c
	c	d
	x	y

Intérêt: Elimination des attributs non désirés et suppression des lignes en double;

Langage LMD

Projection

La projection consiste à extraire **toutes lignes d'une table** pour un sous ensemble de colonnes. Elle donne toutes les lignes pour une partie des colonnes de la table :

Cin	Nom	CodeE
B2511	Asimi	tdi1Info
C2133	Khalid	tdi1Info
D1254	Said	tdi1Info

SELECT distinct cin, nom FROM Etudiant;

Cin	Nom
B2511	Asimi
C2133	Khalid
D1254	Said

Langage LMD

Restriction ou Sélection

La sélection consiste à **extraire d'une relation les occurrences (lignes)** satisfaisant au(x) critère(s) de sélection (le critère de sélection décrit par la clause **WHERE**).

Syntaxe : Nom_Relation_Sel(critère(s)) ;

Le critère de qualification (Q) est défini à base de { **=, <, >=, >, <=, !=** };

Soit R1 une relation,

R1	A	B	C
	a	c	10
	c	d	15
	x	y	20
	c	d	35
	x	y	60

Restriction (ou sélection) de la relation R1
par une qualification $Q = (C < 30)$.

$T = S_{(C < 30)}(R)$

T	A	B	C
	a	c	10
	c	d	15
	x	y	20

Langage LMD

Opérateurs de comparaison

❖ Restriction :

❖ Arithmétique (=, <, >, <>, >=, <=)

- = : Egal à;
- <> : Différent de;
- < : Inférieur;
- > : Supérieur;
- <= : Inférieur ou égal;
- >= : Supérieur ou égal;

❖ Exemple: Trouver tous les clients qui ont la catégorie égal à 'B2';

select distinct * from CLIENT where CAT = ' B2' ;

Langage LMD

Restriction ou Sélection

Elle permet de retenir les lignes répondant à une condition de sélection. La condition est exprimée à l'aide des opérateurs:

- Comparaison: =, >, <, >=, <=, <>;
- Logiques de base: **or**, **and** et **not**;

Cin	Nom	Codee
B2511	Asimi	tdi1Info
C2133	Khalid	tdi1Info
D1254	Said	tdi1Math

SELECT distinct * FROM Etudiant WHERE CodeE='tdi1Info';

Cin	Nom	CodeE
B2511	Asimi	tdi1Info
C2133	Khalid	tdi1Info

Langage LMD

Produit cartésien

Le produit cartésien permet la combinaison des enregistrements de deux relations sans tenir aucun compte du contenu des données. Les relations n'ont donc pas besoin d'avoir la même structure. Le caractère représentant le produit cartésien est « **X** ».

Exemple :

famiLivre(père, mère, fils, Titre, auteur, éditeur) = famille(père, mère, fils) X livre(Titre, auteur, éditeur) .

Syntaxe :

SELECT * FROM *relation_1, relation_2*;

En SQL, cette opération s'écrit simplement:

SELECT * FROM *famille, livre ;*

Langage LMD

Produit cartésien

- Soient R1 et R2 deux relations de schémas distincts,

R1	A	B	C	R2	M	N	P
	a	c	10		far	10	150
	c	d	15		car	15	1000
	x	y	20		min	8	1200

- **Produit cartésien** $T = (R1 * R2)$
ou bien $T = \text{product}(R1, R2)$

T	A	B	C	M	N	P
	a	c	10	far	10	150
	a	c	10	car	15	1000
	a	c	10	min	8	1200
	c	d	15	far	10	150
	c	d	15	car	15	1000
	c	d	15	min	8	1200
	x	y	20	far	10	150
	x	y	20	car	15	1000
	x	y	20	min	8	1200

Langage LMD

Produit cartésien

Le produit cartésien de deux tables permet d'obtenir toutes les combinaisons possibles de lignes de ces deux tables:

SELECT * FROM Etudiant, Filiere;

Etudiant

Cin	Nom	CodeE
B2511	Asimi	tdi1Info
C2133	Khalid	tdi1Info
D1254	Said	tdi1Info



Filiere

CodeE	NomF
tdi1Info	Info
tdi1Phys	Phys



Cin	Nom	Codee	CodeE	NomF
B2511	Asimi	tdi1Info	tdi1Info	Info
C2133	Khalid	tdi1Info	tdi1Info	Info
D1254	Said	tdi1Info	tdi1Info	Info
B2511	Asimi	tdi1Info	tdi1Phys	Phys
C2133	Khalid	tdi1Info	tdi1Phys	Phys
D1254	Said	tdi1Info	tdi1Phys	Phys

Langage LMD

Jointure Naturelle

La jointure consiste à créer une nouvelle table à partir de deux tables **ayant un champ commun (attribut)** et vérifiant un critère de jointure. La **troisième table** contient toutes les lignes obtenues en concaténant les lignes vérifiant **la condition de rapprochement**.

La **jointure** permet d'extraire des données issues de **plusieurs tables**:

- Elle met en **relation deux tables** sur la base d'une **clause** de **jointure** (**comparaison** de colonnes).
- Cette **comparaison** fait **intervenir** une **clé étrangère** de la table **fil** avec une **clé primaire** de la table **père**.
- **Symbole: jointure naturelle** $T = (R1 \bowtie R2)$

Syntaxe:

```
SELECT Col1, Col2...  
FROM nomPere, nomFils ...  
WHERE (conditionsDeJointure);
```

Langage LMD

Jointure Naturelle

Selon l'opérateur, nous distinguons deux types de jointures:

- **Équi-jointure:** Si l'Opérateur de comparaison est = ;
- **Inéqui-jointure:** Si l'Opérateur de comparaison inclut dans cette liste (>, <, >=, <=, <>) ;
- Soient R1 et R2 deux relations de schémas distincts,

R1	A	B	C
	a	c	10
	c	d	15
	x	y	20

R2	M	C	P
	far	10	150
	car	15	1000
	min	8	1200

T	A	B	C	M	P
	a	c	10	far	150
	c	d	15	car	1000

SELECT * FROM relation_1, relation_2 WHERE relation_1.KF = relation_2.KP

Langage LMD

Jointure Naturelle (Équi-jointure)

Une **équi-jointure** est une jointure dont la qualification est une égalité entre deux colonnes.

❖ **SELECT * FROM** Etudiant, Filiere
WHERE Etudiant.**CodeE** = Filiere.**CodeE**

❖ **SELECT * FROM** Etudiant **INNER JOIN** Filiere
ON Etudiant.**CodeE** = Filiere.**CodeE**

Équi-jointure:

Inéqui-jointure:

Cin	Nom	CodeE	CodeE	NomF
B2511	Asimi	tdi1Info	= tdi1Info	Info
C2133	Khalid	tdi1Info	= tdi1Info	Info
D1254	Said	tdi1Info	= tdi1Info	Info
B2511	Asimi	tdi1Info	≠ tdi1Phys	Phys
C2133	Khalid	tdi1Info	≠ tdi1Phys	Phys
D1254	Said	tdi1Info	≠ tdi1Phys	Phys

Langage LMD

Opérateurs et Notations

Opérateur	Sémantique	Notation textuelle	Notation graphique
Restriction	Sélectionner des lignes vérifiant un condition;	$T \leftarrow \sigma_{\text{cond}}(R)$	
Projection	Sélectionner des colonnes;	$T \leftarrow \Pi_{\text{attributs}}(R)$	
Union	Fusionner des lignes de deux tables;	$T \leftarrow R \cup S$	
Intersection	Chercher l'ensemble des lignes communs à deux tables;	$T \leftarrow R \cap S$	
Différence	Chercher les lignes différentes entre deux tables;	$T \leftarrow R - S$	
Produit cartésien	Concaténer chaque ligne de la 1 ^{ère} table avec toutes les lignes de la 2 ^{ème} tables;	$T \leftarrow R \times S$	
Jointure	Etablir le lien sémantique entre les tables;	$T \leftarrow R \bowtie_{\text{condition}} S$	

Langage LMD

Opérateurs de comparaison

La clause **LIKE** permet la construction des conditions de comparaison à base d'une chaîne de caractères:

- ❖ **'_'** remplace n'importe quel caractère;
- ❖ **'%'** remplace n'importe quelle chaîne de caractères;
- ❖ **Exemple 1:** Quels sont les clients qui ont une catégorie commence par le caractère **C** ?
 - SQL : **SELECT ***
FROM Client
WHERE CAT like 'C%';
- **Exemple 2:** Quels sont les noms de Clients possédant un **'O'** en seconde position ?
 - SQL : **SELECT ***
FROM Client
WHERE Nom like '_O%';

Langage LMD

Opérateurs de comparaison

La clause **BETWEEN** et **NOT BETWEEN** définit un interval des valeurs possibles:

❖ **Exemple** : Quels sont les clients qui ont un montant entre 0 et 1300 ?

- SQL : **SELECT Nom, COMPTE**

FROM Client

WHERE COMPTE BETWEEN 0 AND 1300;

Langage LMD

Opérateurs de comparaison

Les clauses **In** et **not In** permettent la construction d'appartenance à **une liste des valeurs**:

❖ Exemple 1 : Quels sont les clients qui ont la catégorie ('B1','B2','B3')?

- SQL : **SELECT ***
FROM Client
WHERE CAT in ('B1','B2','B3');

❖ Exemple 2 : Quels sont les clients qui n'ont pas la Localité Toulouse et Namur?

- SQL : **SELECT ***
FROM Client
WHERE LOCALITE not in ('Toulouse','Namur');

Langage LMD

Opérateurs de comparaison

En règle générale, les données **NULL** représentent des données inexistantes, des données manquantes ou des **données inconnues**.

- ❖ **IS NULL** et **IS NOT NULL** est utilisé avec une clause **WHERE** dans les **instructions/requêtes SELECT, UPDATE et DELETE** pour valider si la colonne a une valeur ou s'il n'existe pas de données pour cette colonne.

❖ Remarques:

- ❖ il faut noter que **NULL** et **0** ne sont pas identiques.
- ❖ Une colonne avec la valeur **NULL** n'a pas de valeur, elle est vide.

❖ Syntaxes:

- ❖ **SELECT** column_name1, column_name2, etc **FROM** table_name **WHERE** column_name1 **IS NULL**;
- ❖ **SELECT** column_name1, column_name2, etc **FROM** table_name **WHERE** column_name1 **IS NOT NULL**;

Langage LMD

Opérateurs de comparaison

❖ Exemple 1 : Quels sont les clients qui ont l'attribut Localité non renseigné?

- SQL : **SELECT ***
FROM Client
WHERE CAT IS NULL;

❖ Exemple 2 : Quels sont les clients qui ont l'attribut Nom renseigné?

- SQL : **SELECT ***
FROM Client
WHERE NOM IS Not NULL;

Langage LMD

Clause ORDER BY

La clause **ORDER BY** permet de trier les données sélectionnées dans une table. Elle **trie les données dans un ordre croissant ou bien décroissant** en utilisant une ou plusieurs colonnes.

❖ Par défaut, les résultats de la requête sont triés par **ordre croissant**.

❖ Syntaxe:

```
SELECT attribut1, attribut2, ... FROM Nom_table ORDER BY attribut1 [ASC],  
attribut2[DESC], ... ;
```

❖ **Exemple** : Cherchez les clients qui ont une Localité égale à Toulouse et trie selon sa catégorie?

- SQL :

```
SELECT * FROM Client  
WHERE Localite= 'Toulouse '  
ORDER BY CAT DESC;
```

Langage LMD

Conditions Complexes

Les opérateurs **AND**, **OR** en SQL sont utilisés avec les instructions / requêtes **SELECT**, **UPDATE** et **DELETE** pour:

- ❖ sélectionner, mettre à jour et supprimer des enregistrements particuliers dans une table.
- ❖ Ils remplissent les conditions spécifiées dans la clause **WHERE** :
 - ❖ **SELECT** column_name1, column_name2, etc
FROM table_name
WHERE [condition1] **AND** [condition2] **AND** [condition3] etc;
 - ❖ **SELECT** column_name1, column_name2, etc
FROM table_name
WHERE [condition1] **OR** [condition2] **OR** [condition3] etc;
 - ❖ **SELECT** column_name1, column_name2, etc
FROM table_name
WHERE [condition1] **AND** [condition2] **OR** [condition3] etc;

Langage LMD

Conditions Complexes

- ❖ Lors de l'évaluation d'une expression complexe:
 - ❖ la négation (not) a priorité sur la conjonction (and),
 - ❖ la conjonction a priorité sur la disjonction (or).
- ❖ Plutôt, nous pouvons imposer un ordre d'évaluation différent à l'aide de parenthèses:
 - ❖ $LOCALITE = 'Toulouse' \text{ and } COMPTE < 0 \text{ or } CAT = 'C1'$;
- ❖ Cette condition est interprétée comme suite:
 - ❖ si $(LOCALITE = 'Toulouse' \text{ et } COMPTE < 0)$ ou si $(CAT = 'C1')$ ou si les deux conditions sont vérifiées.
- ❖ Elle est équivalente :
 - ❖ $(LOCALITE = 'Toulouse' \text{ and } COMPTE < 0) \text{ or } CAT = 'C1'$;
- ❖ En revanche, l'expression :
 - ❖ $LOCALITE = 'Toulouse' \text{ and } (COMPTE < 0 \text{ or } CAT = 'C1')$;
- ❖ Cette condition est vérifié si les deux sous-conditions sont **simultanément vérifiées**;

Langage LMD

Conditions Complexes

❖ **Exemple 1:** Quels sont les clients qui ont l'attribut Localité non renseigné et des comptes supérieurs à zéro?

- SQL : **SELECT ***
FROM Client
WHERE CAT IS NULL **AND** COMPTE >0;

❖ **Exemple 2:** Quels sont les clients qui ont une catégorie B1 ou bien une Localité égale à Toulouse?

- SQL : **SELECT ***
FROM Client
WHERE CAT='B1' **OR** LOCALITE='Toulouse';

Langage LMD

Fonctions

Une fonction est un ensemble d'instructions combinées pour effectuer une tâche. SQL définit plusieurs fonctions intégrées permettant d'effectuer de nombreuses tâches arithmétiques:

- ❖ Fonctions d'Agrégat;
- ❖ Fonctions scalaires;
- ❖ Fonction GROUP BY;
- ❖ Fonction HAVING;
- ❖ Fonctions d'agrégat:
 - ❖ Les fonctions d'agrégat sont des fonctions qui renvoient une sortie unique en calculant les valeurs à partir de la table / colonne:
 - ❖ **sum(), count(), avg (), min(), max (), first (), last (), etc.**
- ❖ Fonctions Scalaires:
 - ❖ Les fonctions scalaires renvoient une sortie unique à partir de la valeur d'entrée donnée:
 - ❖ **ucase(), lcase (), length(), round(), mid(), format (), now(), etc.**

Langage LMD

Fonctions d'Agrégat

Fonction	Description
SUM()	Elle compte et renvoie la somme d'une colonne de type numérique.
COUNT()	Elle compte et renvoie le nombre de lignes d'une requête qui vérifie un critère de recherche.
AVG()	Elle compte et renvoie la valeur moyenne d'une colonne de type numérique.
MIN()	Elle renvoie la valeur minimale d'une colonne. La colonne de type numérique.
MAX()	Elle renvoie la valeur maximale d'une colonne de type numérique.
FIRST()	Elle renvoie la première valeur de la colonne donnée.
LAST()	Elle renvoie la dernière valeur de la colonne donnée.

Langage LMD

Fonctions d'Agrégat

❖ **Exemple 1** : Donner le total des salaires des clients qui ont des comptes supérieurs à zéro:

- SQL : **SELECT SUM**(COMPTE)
FROM Client
WHERE COMPTE >0;

❖ **Exemple 2** : Donner le total des salaires des clients qui ont une catégorie B1 ou bien une Localité égale à Toulouse:

- SQL : **SELECT SUM**(COMPTE)
FROM Client
WHERE CAT = 'B1' OR LOCALITE = 'Toulouse';

Langage LMD

Fonctions d'Agrégat

❖ **Exemple 1** : Calculer le nombre des clients qui ont l'attribut Localité non renseigné et des Comptes supérieurs à zéro:

- SQL : **SELECT COUNT(*)**
FROM Client
WHERE CAT IS NULL AND COMPTE >0;

❖ **Exemple 2** : Calculer le nombre des clients qui ont une Catégorie B1 ou bien une Localité égale à Toulouse:

- SQL : **SELECT COUNT(*)**
FROM Client
WHERE CAT = 'B1' OR LOCALITE = 'Toulouse';

Langage LMD

Fonctions d'Agrégat

❖ **Exemple 1** : Calculer la moyenne du salaire des clients qui ont une catégorie C1:

- SQL : **SELECT** **AVG** (**COMPTE**)
FROM Client
WHERE CAT='C1';

❖ **Exemple 2** : Calculer la moyenne du salaire des clients qui ont une Catégorie B2 et une Localité égale à Namur:

- SQL : **SELECT** **AVG** (**COMPTE**)
FROM Client
WHERE CAT='B1' **AND** LOCALITE='Namur';



Langage LMD

Fonctions d'Agrégat

❖ **Exemple 1** : Trouver le salaire minimal et maximal des clients qui ont un salaire renseigné:

- SQL : **SELECT** MAX(COMPTE), MIN(COMPTE)
FROM Client
WHERE COMPTE IS Not NULL;

❖ **Exemple 2** : Trouver le salaire minimal et maximal des clients qui ont une Catégorie B2 et des Comptes supérieurs à zéro :

- SQL : **SELECT** MAX(COMPTE), MIN(COMPTE)
FROM Client
WHERE CAT='B1' AND COMPTE>0;

Langage LMD

Fonctions Scalaires

Fonction	Description
LENGTH()	Elle renvoie la longueur totale des caractères de chaque ligne d'une colonne donnée.
MID()	Elle extrait les caractères d'un champ de texte.
ROUND()	Elle arrondit une valeur numérique au nombre entier le plus proche.
LCASE()	Elle convertit une colonne donnée en minuscule.
UCASE ()	Elle convertit une colonne donnée en MAJUSCULE.
NOW()	Elle renvoie la date et l'heure actuelles du système.
FORMAT()	Elle est utilisée pour formater une valeur de champ/colonne de la manière dont elle devrait être affichée dans la sortie.
RAND()	Elle régénère des nombres aléatoires.
SQRT()	Elle est utilisée pour trouver la valeur de la racine carrée du nombre donné.
CONCAT()	Elle concatène 2 chaînes ou plus et forme une seule chaîne.

Langage LMD

Fonctions Scalaires

La Fonction Length() renvoie la longueur totale d'une colonne donnée. Parmi ces caractéristiques, nous citons:

- ❖ Elle ne compte pas les espaces de fin à la fin d'une chaîne lors du calcul de la longueur de la chaîne.
- ❖ Elle compte les espaces au début de la chaîne lors du calcul de la longueur de la chaîne.
- ❖ Elle retourne **NULL** si la chaîne est **NULL**.
- ❖ La syntaxe de la **fonction SQL Length ()** est:
 - **SELECT Length(column_name) FROM table_name;**
- ❖ **Exemple 1** : Ecrire une requête qui affiche la taille des noms des clients de la table Client:
 - **SQL : SELECT Length(NOM)
FROM Client;**

Langage LMD

Fonctions Scalaires

La Fonction **MID()** extrait une **sous-chaîne** d'une chaîne :

❖ La syntaxe de cette fonction est:

- **SELECT MID("chaîne", start, length) AS ChainExtrait;**

❖ **Exemple 1:** Ecrire une requête qui extrait une sous-chaine (5 caractères de la position 6) d'une chaîne de caractères:

- SQL : **SELECT MID(" ASIMI Younes", 6, 5) AS ChainExtrait;**

❖ Nous pouvons utiliser cette fonction à fin d'extraire une sous-chaine des caractères d'un colonne d'une table donnée:

❖ La syntaxe de cette fonction est:

- **SELECT MID(column_name, start, length)FROM table_name;**

❖ **Exemple 2:** Ecrire une requête qui extrait une sous-chaine (6 caractère de la position 3) de la colonne Nom de la table Client:

- SQL : **SELECT MID(NOM, 3, 6) AS ChainExtrait FROM Client;**

Langage LMD

Fonctions Scalaires

La **Fonction ROUND()** renvoie un nombre arrondi à un certain nombre de décimales.

❖ La syntaxe de cette fonction est:

- **SELECT ROUND(column_name, decimals) FROM table_name;**

❖ **column_name:** C'est le nom de la colonne dont la valeur doit être arrondie. C'est un champ obligatoire.

❖ **Decimals:** Il spécifie le nombre de décimales à retourner. C'est un champ obligatoire.

❖ **Exemple 1 :** Ecrire une requête qui affiche le salaire entier des clients de la table Client:

- SQL : **SELECT ROUND(COMPTE, 0)**
FROM Client;

Langage LMD

Fonctions Scalaires

La **Fonction LCASE()** convertit les caractères d'une colonne donnée en minuscule.

❖ La syntaxe de cette fonction est:

- **SELECT LCASE(column_name) FROM table_name;**

❖ **Exemple 1** : Ecrire une requête qui affiche les noms des clients de la table Client en minuscule :

- SQL : **SELECT LCASE(NOM)**

FROM Client;

❖ **Rq: même principe que la fonction UCASE** qui convertit les caractères en majuscule;

Langage LMD

Fonctions Scalaires

La Fonction **NOW()** retourne la date et l'heure actuelles du système. Elle pourra être utilisée conjointement dans la fonction DATETIME() ou bien DATE() pour enregistrer la date d'ajout ou de modification d'une donnée. Cela permet de savoir exactement le moment où a été ajoutée une donnée contenue dans une table.

❖ La syntaxe de cette fonction est:

• **SELECT NOW();**

❖ Supposant que nous voulons savoir la date d'ajout ou bien de modification de la quantité de stock de chaque produit. Nous ajoutons une colonne **Date_Ajout** à la table produit. Les champs de cette table deviennent:

- **NumProd:** Numéro du produit;
- **NomProd:** Nom du produit;
- **PrixProdUnit:** Prix Unitaire de produit;
- **TVA:** Taxe sur la valeur ajoutée, il dépend de type de produit: C1→ TVA=20/100, B1→ TVA= 16/100;

Langage LMD

Fonctions Scalaires

- **QuantProd:** Quantité stockée;
 - **TypeProd:** Type de produit C1 ou bien B1;
 - **DateAjout:** la date d'ajout d'un produit;
- ❖ **Exemple 2 :** Ecrire une requête qui ajoute pour chaque enregistrement la date et l'heure d'ajout d'un produit donné à la la table Client:
- SQL : **INSERT INTO Produit(NumProd, NomProd, PrixProdUnit, TVA, QuantProd, TypeProd, DateAjout) VALUES ('BC23', 'Moteur', 123, 0.2, 23, 'C1', NOW());**
- ❖ Nous pouvons aussi combiner cette fonction avec la fonction date() pour afficher juste la date:
- SQL : **SELECT DATE(NOW());**

Langage LMD

Fonctions Scalaires

La Fonction FORMAT() est utilisée pour formater une valeur de champ / colonne indiquant comment elle devrait être affichée dans la sortie. En général, nous utilisons cette fonction pour afficher les dates dans un format précis:

- La syntaxe de cette fonction est:

- **SELECT FORMAT(column_name, format) FROM table_name;**

❖ Avec :

- **column_name:** C'est le nom de la colonne qui doit être formatée. C'est un champ obligatoire.
- **Format:** Il spécifie le format. C'est un champ obligatoire.

❖ **Exemple 1:**

- **SELECT FORMAT('15/12/2018', 'MM, DD, yyyy');**
- **SELECT FORMAT('113456777', '###-##-#####');**



Langage LMD

Fonctions Scalaires

- ❖ **Exemple 2** : Ecrire une requête qui affiche les noms, les salaires et la date actuelle en format '**DD-MM-YYYY**' de tous les clients:
 - SQL : **SELECT NOM, COMPTE, FORMAT(Now(),'DD-MM-YYYY') AS NOW_DATE
FROM Client;**

Langage LMD

Fonctions Scalaires

La Fonction RAND() renvoie un nombre aléatoire à virgule, compris **entre 0 et 1**. Le résultat de cette fonction sera différent à chaque fois que la fonction est exécutée dans une requête SQL.

❖ La syntaxe de cette fonction est:

❖ **SELECT RAND();**

❖ **Exemple 1** : Nous pouvons utiliser cette fonction pour régénérer des nombres aléatoires:

- SQL : `SELECT RAND() * 1000;`

Langage LMD

Fonctions Scalaires

La Fonction **CONCAT()** permet de concaténer les valeurs de plusieurs colonnes pour ne former qu'une **seule chaîne de caractère**.

❖ La syntaxe de cette fonction est:

- **CONCAT(colonne1, colonne2,...)**

❖ **Exemple 1** : Créer une table nommée Utilisateur qui contient les champs suivants:

- **Id**: identifiant de l'utilisateur;
- **Nom**: Nom de l'utilisateur;
- **Prenom**: Prénom de l'utilisateur;
- **Now_Date**: Date d'ajout d'un utilisateur;

❖ Ecrire une requête qui affiche les informations des utilisateurs en concaténant les colonnes Nom et Prénom séparés par un espace :

- SQL : **SELECT id, CONCAT(prenom, ' ', nom) AS NomComplet, Now_Date FROM utilisateur;**

Langage LMD

Fonction GROUP BY

Elle est utilisée avec les instructions SELECT pour **regrouper** des données similaires dans le jeu de résultats.

❖ La syntaxe de cette fonction est:

- SQL : **SELECT** colonne1, fonction(colonne2) **FROM** table **GROUP BY** colonne1;

❖ Cette commande doit toujours s'utiliser après la commande **WHERE** et avant la commande **HAVING**;

❖ Pour obtenir la moyenne total de chaque client en regroupant les commandes des mêmes Noms :

- ❖ SQL : **SELECT** Nom, **AVG**(COMPTE) **FROM** Client **GROUP BY** Nom;

❖ La fonction **AVG()** calcule la moyenne des salaires pour des clients qui ont le même Nom.

Langage LMD

Fonction HAVING

Cette Fonction est presque similaire à WHERE à la seule différence que HAVING permet le filtrage de données à base des fonctions d'agrégation telles que: **SUM()**, **COUNT()**, **AVG()**, **MIN()** ou **MAX()**.

- ❖ L'objet principal de la fonction HAVING () est que les fonctions agrégées ne peuvent pas être utilisées dans la clause **where**. Ils sont donc utilisés dans une clause **where** à l'aide de la fonction **HAVING ()**.
- ❖ La syntaxe de cette fonction est:
 - **SELECT colonne1, Fonction(colonne2) FROM nom_table GROUP BY colonne1**
HAVING Fonction(colonne2) operateur valeur;
- ❖ Cette requête permet donc de sélectionner les colonnes d'une **table** donnée en **regroupant** les lignes qui ont des valeurs identiques sur la colonne "**colonne1**" et que la **condition** de **HAVING** soit respectée.
- ❖ Rq: HAVING est très souvent utilisé en même temps que GROUP BY bien que ce ne soit pas obligatoire.

Langage LMD

Fonction HAVING

- ❖ **Exemple 1:** Créer une table nommée **Commande** qui contient les champs suivants:
 - ❖ **Id:** identifiant de la commande
 - ❖ **Nom:** Utilisateur qui as commande une telle commande;
 - ❖ **Prix_Command:** Prix total de la commande demandée par un tel utilisateur;
 - ❖ **Date_Command:** Date d'enregistrement d'une telle commande;
- ❖ Ecrire une requête qui affiche les informations (**id**, **Nom**, somme de prix commandé) des clients qui ont commandé plus de 200 DH :
 - SQL :

```
SELECT id, Nom, SUM(Prix_Command)
FROM Commande
GROUP BY id, Nom
HAVING SUM(Prix_Command) > 200;
```

Langage LMD

Fonction HAVING

- ❖ Exemple 1 : Créer une table nommée **Clients** qui contient les champs suivants:
 - ❖ **CID** : identifiant du client;
 - ❖ **CNom** : Nom du client;
 - ❖ **Ville** : Ville du client;
 - ❖ **Pays** : Pays du client;
- ❖ Ecrire une requête qui compte et affiche, le pays et le nombre, de clients dans chaque pays. N'incluez que les pays comptant plus de 3 clients: :
 - SQL :

```
SELECT COUNT(CID), Pays
FROM Clients
GROUP BY Pays
HAVING COUNT(CID) > 3;
```
- ❖ Classez les pays selon l'ordre décroissante de clients comptés:

```
ORDER BY COUNT(CID) DESC;
```